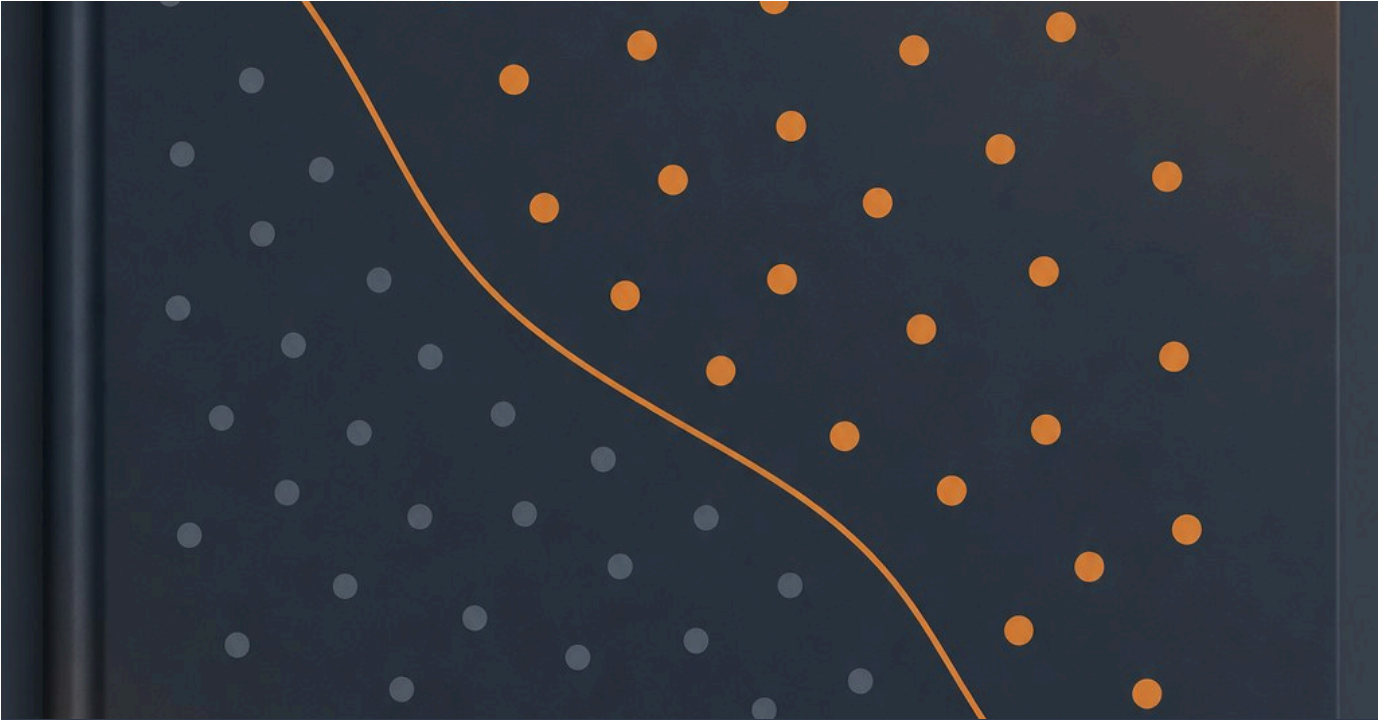




FORMATION IA - MACHINE LEARNING

Machine Learning

Supervise, metriques, overfitting et pipelines : les bases du ML sans jargon opaque.

DanielCraft - Livre debutant

Sommaire

Clique un chapitre ou un sous-chapitre pour y aller.

1. Chapitre 1 - Salut, c'est quoi le machine learning ?

- [Ce que ce livre va faire](#)
- [Machine learning vs IA generative](#)
- [Fil rouge](#)
- [Erreur classique](#)
- [A toi](#)
- [Où le ML se cache dans ta vie](#)
- [Ce que "apprendre" veut dire ici](#)
- [Developpement : penser comme un artisan des modeles](#)
- [Donnees : le personnage principal](#)
- [Mise en production \(aperçu\).](#)
- [Culture et communication](#)
- [Pour aller plus loin sans te perdre](#)

2. Chapitre 2 - Apprentissage supervise : apprendre avec les bonnes reponses

- [Deux grandes familles](#)
- [Le deroulement mental](#)
- [Exemple Noe](#)
- [Limites](#)
- [Erreur classique](#)
- [A toi](#)
- [Qualite des labels](#)
- [Volume : combien faut-il ?](#)
- [Developpement : penser comme un artisan des modeles](#)
- [Donnees : le personnage principal](#)
- [Mise en production \(aperçu\).](#)
- [Culture et communication](#)
- [Pour aller plus loin sans te perdre](#)

3. Chapitre 3 - Apprentissage non supervise : trouver des structures sans label

- [Clustering : regrouper](#)
- [Quand s'en servir](#)
- [Attention aux illusions](#)
- [Lien avec le reste de l'IA](#)
- [Erreur classique](#)
- [A toi](#)
- [Valider un clustering sans label magique](#)
- [Developpement : penser comme un artisan des modeles](#)
- [Donnees : le personnage principal](#)
- [Mise en production \(aperçu\).](#)
- [Culture et communication](#)
- [Pour aller plus loin sans te perdre](#)

4. Chapitre 4 - Regression : predire un nombre

- [Regression lineaire : l'intuition](#)
 - [Au-dela de la droite](#)
 - [Erreurs et unites](#)
 - [Pieges](#)
 - [Erreur classique](#)
 - [A toi](#)
 - [Interpretable d'abord](#)
 - [Developpement : penser comme un artisan des modeles](#)
 - [Donnees : le personnage principal](#)
 - [Mise en production \(aperçu\)](#)
 - [Culture et communication](#)
 - [Pour aller plus loin sans te perdre](#)
5. [Chapitre 5 - Classification : predire une classe](#)
- [Frontiere de decision](#)
 - [Scores et seuils](#)
 - [Classes desequilibrees](#)
 - [Multclasse](#)
 - [Erreur classique](#)
 - [A toi](#)
 - [Probabilites calibrees \(idee\)](#)
 - [Developpement : penser comme un artisan des modeles](#)
 - [Donnees : le personnage principal](#)
 - [Mise en production \(aperçu\)](#)
 - [Culture et communication](#)
 - [Pour aller plus loin sans te perdre](#)
6. [Chapitre 6 - Arbres de decision : des questions en cascade](#)
- [Pourquoi c'est pedagogique](#)
 - [Limites d'un arbre seul](#)
 - [Regles de bonne vie](#)
 - [Erreur classique](#)
 - [A toi](#)
 - [Forets et boosting \(aperçu\)](#)
 - [Developpement : penser comme un artisan des modeles](#)
 - [Donnees : le personnage principal](#)
 - [Mise en production \(aperçu\)](#)
 - [Culture et communication](#)
 - [Pour aller plus loin sans te perdre](#)
7. [Chapitre 7 - Features : ce que le modele "voit"](#)
- [Types courants](#)
 - [Disponible au bon moment](#)
 - [Nettoyage et preparation](#)
 - [Feature engineering](#)
 - [Erreur classique](#)
 - [A toi](#)

- [Documentation des features](#)
- [Developpement : penser comme un artisan des modeles](#)
- [Donnees : le personnage principal](#)
- [Mise en production \(aperçu\)](#)
- [Culture et communication](#)
- [Pour aller plus loin sans te perdre](#)

8. [Chapitre 8 - Train / test : apprendre sans se mentir](#)

- [Comment separer](#)
- [Validation croisee \(idee\)](#)
- [Fuite \(data leakage\)](#)
- [Ritual DanielCraft](#)
- [Erreur classique](#)
- [A toi](#)
- [Simulation de production](#)
- [Developpement : penser comme un artisan des modeles](#)
- [Donnees : le personnage principal](#)
- [Mise en production \(aperçu\)](#)
- [Culture et communication](#)
- [Pour aller plus loin sans te perdre](#)

9. [Chapitre 9 - Overfitting : trop coller a l'entrainement](#)

- [Sous-apprentissage aussi](#)
- [Signes qui doivent t'alerter](#)
- [Remedes pratiques](#)
- [Lien avec l'IA generative](#)
- [Erreur classique](#)
- [A toi](#)
- [Regularisation en mots simples](#)
- [Developpement : penser comme un artisan des modeles](#)
- [Donnees : le personnage principal](#)
- [Mise en production \(aperçu\)](#)
- [Culture et communication](#)
- [Pour aller plus loin sans te perdre](#)

10. [Chapitre 10 - Metriques : mesurer sans se raconter d'histoires](#)

- [Regression](#)
- [Classification](#)
- [Coût métier > score unique](#)
- [Baseline toujours](#)
- [Erreur classique](#)
- [A toi](#)
- [Raconter un score a un non-tech](#)
- [Developpement : penser comme un artisan des modeles](#)
- [Donnees : le personnage principal](#)
- [Mise en production \(aperçu\)](#)
- [Culture et communication](#)

- [Pour aller plus loin sans te perdre](#)

11. [Chapitre 11 - Pipeline : enchaîner proprement les étapes](#)

- [Étapes typiques](#)
- [Pourquoi scikit-learn aime ça](#)
- [Reproductibilité](#)
- [Monitoring après déploiement](#)
- [Erreur classique](#)
- [A toi](#)
- [Tests du pipeline](#)
- [Développement : penser comme un artisan des modèles](#)
- [Données : le personnage principal](#)
- [Mise en production \(aperçu\)](#)
- [Culture et communication](#)
- [Pour aller plus loin sans te perdre](#)

12. [Chapitre 12 - Biais dans les données : le modèle hérite](#)

- [Types de biais utiles à connaître](#)
- [Exemple Noé](#)
- [Que faire concrètement](#)
- [Lien légal et moral](#)
- [Erreur classique](#)
- [A toi](#)
- [Processus de revue](#)
- [Développement : penser comme un artisan des modèles](#)
- [Données : le personnage principal](#)
- [Mise en production \(aperçu\)](#)
- [Culture et communication](#)
- [Pour aller plus loin sans te perdre](#)

13. [Chapitre 13 - Scikit-learn : l'idée sans jargon opaque](#)

- [La grammaire mentale](#)
- [Pipeline dans scikit-learn](#)
- [Ce que scikit-learn fait bien](#)
- [Mini parcours d'apprentissage code \(optionnel\)](#)
- [Erreur classique](#)
- [A toi](#)
- [Écosystème voisin](#)
- [Développement : penser comme un artisan des modèles](#)
- [Données : le personnage principal](#)
- [Mise en production \(aperçu\)](#)
- [Culture et communication](#)
- [Pour aller plus loin sans te perdre](#)

14. [Chapitre 14 - À retenir](#)

- [Boucle DanielCraft](#)
- [Ce que tu peux oublier](#)

- [A toi](#)
- [Checklist une minute](#)
- [Developpement : penser comme un artisan des modeles](#)
- [Donnees : le personnage principal](#)
- [Mise en production \(aperçu\)](#)
- [Culture et communication](#)
- [Pour aller plus loin sans te perdre](#)

15. [Chapitre 15 - Atelier : une regression de bout en bout \(sur papier ou code\)](#)

- [Etapas](#)
- [Livrable](#)
- [Conseil](#)
- [Prolongement](#)
- [Developpement : penser comme un artisan des modeles](#)
- [Donnees : le personnage principal](#)
- [Mise en production \(aperçu\)](#)
- [Culture et communication](#)
- [Pour aller plus loin sans te perdre](#)

16. [Chapitre 16 - Atelier : classification et cout des erreurs](#)

- [Etapas](#)
- [Livrable](#)
- [Piege a eviter](#)
- [Prolongement](#)
- [Developpement : penser comme un artisan des modeles](#)
- [Donnees : le personnage principal](#)
- [Mise en production \(aperçu\)](#)
- [Culture et communication](#)
- [Pour aller plus loin sans te perdre](#)

17. [Chapitre 17 - Atelier : dessiner et valider un pipeline](#)

- [Etapas](#)
- [Livrable](#)
- [Conseil DanielCraft](#)
- [Prolongement](#)
- [Developpement : penser comme un artisan des modeles](#)
- [Donnees : le personnage principal](#)
- [Mise en production \(aperçu\)](#)
- [Culture et communication](#)
- [Pour aller plus loin sans te perdre](#)

18. [Chapitre 18 - Erreurs classiques \(et comment les voir venir\)](#)

- [Les hits](#)
- [Comment se rattraper](#)
- [Mini histoires](#)
- [A toi](#)
- [Exercice miroir](#)

- [Developpement : penser comme un artisan des modeles](#)
- [Donnees : le personnage principal](#)
- [Mise en production \(aperçu\)](#)
- [Culture et communication](#)
- [Pour aller plus loin sans te perdre](#)

19. [Chapitre 19 - Bonnes pratiques ML](#)

- [Avant de coder](#)
- [Pendant](#)
- [Après](#)
- [Culture](#)
- [A toi](#)
- [Ritual de fin de sprint](#)
- [Developpement : penser comme un artisan des modeles](#)
- [Donnees : le personnage principal](#)
- [Mise en production \(aperçu\)](#)
- [Culture et communication](#)
- [Pour aller plus loin sans te perdre](#)

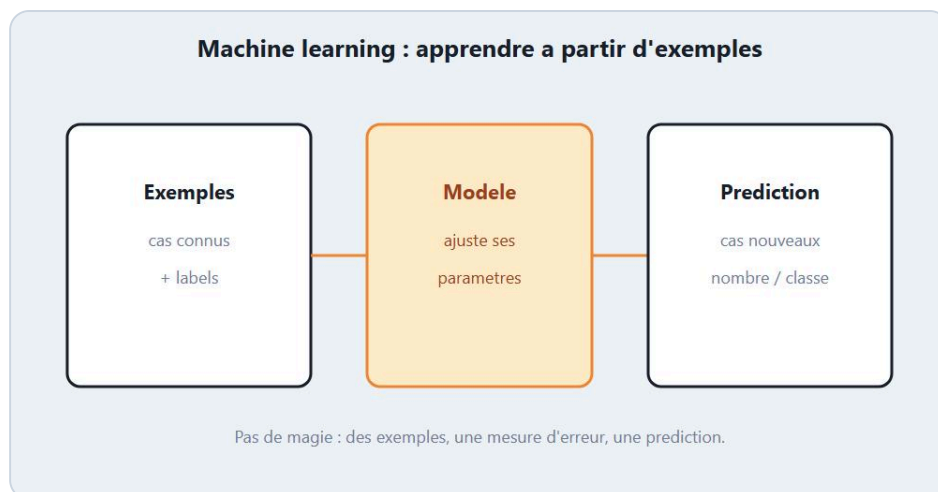
20. [Quiz final](#)

- [Corriges](#)
- [Bonus](#)
- [Developpement : penser comme un artisan des modeles](#)
- [Donnees : le personnage principal](#)
- [Mise en production \(aperçu\)](#)
- [Culture et communication](#)
- [Pour aller plus loin sans te perdre](#)

21. [Chapitre 21 - Bravo](#)

- [La suite](#)
- [Projet 30 jours](#)
- [Developpement : penser comme un artisan des modeles](#)
- [Donnees : le personnage principal](#)
- [Mise en production \(aperçu\)](#)
- [Culture et communication](#)
- [Pour aller plus loin sans te perdre](#)

Chapitre 1 - Salut, c'est quoi le machine learning ?



Machine learning : apprendre a partir d'exemples.

Le **machine learning**, c'est l'art d'apprendre a une machine a partir d'exemples, au lieu d'ecrire a la main toutes les regles. Tu montres des cas. Le modele ajuste ses parametres. Ensuite, face a un cas nouveau, il predit, classe, regroupe, ou estime un nombre. Ce n'est pas de la magie. Ce n'est pas non plus "l'IA qui ecrit des romans". C'est souvent plus discret, et partout : spam, recommandations, detection de fraude, prevision de demande, diagnostic assiste, tri de photos.

Chez DanielCraft, on aime partir d'une image simple. Imagine que tu veux savoir si un mail est indésirable. Au lieu d'ecrire mille regles ("si le mot viagra alors spam"), tu montres des milliers de mails deja labels "spam" ou "pas spam". Le modele cherche des motifs. Puis il score de nouveaux mails. Il se trompera parfois. Ton travail, c'est de mesurer ces erreurs, de choisir les bons exemples, et de savoir quand ne pas faire confiance.

★ A RETENIR

Le machine learning apprend sur des exemples. Il predit, il ne "comprend" pas comme toi.

Ce que ce livre va faire

Tu vas comprendre l'apprentissage **supervise** et non supervise, la regression, la classification, les arbres de decision, le decoupage **train/test**, l'**overfitting**, les **features**, les metriques, l'idee de pipeline, les biais dans les donnees, et l'esprit de scikit-learn sans jargon opaque. Puis un recap, des ateliers, des bonnes pratiques, un quiz, un bravo. Niveau debutant curieux. Pas besoin d'etre data scientist. Besoin d'aimer les exemples concrets.

Machine learning vs IA generative

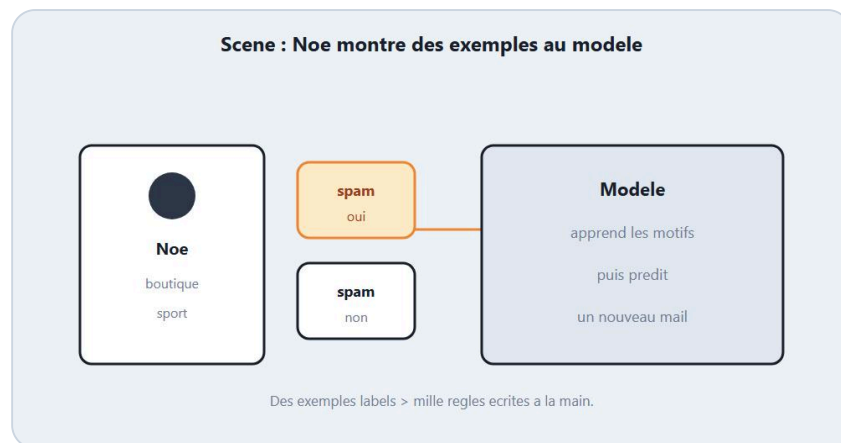
Un LLM genere du texte. Un modele de machine learning "classique" repond souvent a une question etroite : quel prix ? quelle classe ? quel cluster ? Les deux existent en 2026. Ce livre se concentre sur le second, parce que beaucoup de decisions metier reposent encore la-dessus, et parce que comprendre ca t'aide aussi a demystifier l'IA en general.

♦ ASTUCE

Avant de lire plus loin, note une prediction utile dans ton metier (un nombre ou une classe). Tu t'en serviras comme fil rouge.

Fil rouge

On suivra Noe, qui tient une petite boutique en ligne d'equipement sportif. Il veut predire si une commande risque un retour, estimer un volume de ventes, et regrouper ses clients par comportements. Il n'a pas une equipe data. Il a un tableur, un peu de Python plus tard, et surtout besoin d'idees justes. Toi aussi, tu pourras substituer ton contexte.



Exemple : Noe montre des mails labels au modele.

Erreur classique

Croire que "plus de donnees = automatiquement mieux" sans regarder la qualite. Ou croire qu'un score eleve sur les donnees d'entrainement prouve que le modele est bon. Spoiler du chapitre overfitting : non.

⚠ ATTENTION

Un score brillant sur l'entrainement ne prouve rien. Ce qui compte, c'est le comportement sur des cas nouveaux.

A toi

Ecris une prediction utile dans ton monde (nombre ou classe). Note quelles donnees tu aurais, et ce qui se passerait si le modele se trompe. Garde ce papier.

Ou le ML se cache dans ta vie

Ton streaming te propose une serie : un modele a estime une preference. Ta banque bloque une carte : un score d'anomalie a clignote. Ton app photo regroupe des visages : un regroupement / reconnaissance. Ton outil mail range des pourriels : une classification. Rarement tu vois le mot "machine learning" sur le bouton. Souvent tu vois juste un comportement. Ce livre te rend capable de reconnaitre la logique derriere le comportement, et surtout de la construire proprement quand tu en as besoin.

Ce que "apprendre" veut dire ici

Apprendre, ce n'est pas memoriser une leçon scolaire. C'est ajuster des paramètres pour réduire une erreur moyenne sur des exemples, puis espérer que ça tienne sur des exemples nouveaux. Cette espérance n'est pas automatique. Elle se mérite par la qualité des données, le split, les métriques, et l'humilité face au déploiement. Chez DanielCraft, on célèbre les projets qui battent une baseline honnête, pas ceux qui affichent 99,9 % sans contexte.

Developpement : penser comme un artisan des modeles

Le machine learning n'est pas un distributeur de vérité. C'est un artisanat de décisions sous incertitude. Tu choisis une question, tu rassembles des exemples, tu acceptes une erreur moyenne, tu te donnes les moyens de la mesurer, tu decides si cette erreur est tolérable pour le cas d'usage. Beaucoup de frustration vient d'attendre la perfection là où il fallait un score utile avec un humain dans la boucle.

Quand Noé prédit un risque de retour, il ne remplace pas le service client. Il priorise. Quand un hôpital utilise un score (hors du périmètre de ce livre introductif, et avec des cadres stricts), l'enjeu n'est plus le même : les coûts d'erreur explosent, les biais deviennent critiques, la gouvernance monte. Adapte toujours la profondeur de ta démarche à l'impact. Un modèle jouet sur un CSV public n'exige pas la même revue qu'un score qui bloque un crédit.

Donnees : le personnage principal

Les algorithmes changent. Les principes de données restent : définition claire, représentativité, fraîcheur, droits, documentation, absence de fuite, inspection des cas bizarres. Passe plus de temps sur les données que sur le shopping d'algorithmes. C'est le conseil le moins glamour et le plus rentable du livre. Un arbre simple sur des features excellentes bat une usine à gaz sur un tableau sale.

Mise en production (aperçu)

Un notebook n'est pas un produit. En production, tu dois gérer des entrées manquantes nouvelles, des catégories inconnues, des délais, des journaux, des versions, des rollback, des alertes si la métrique chute. Tu n'as pas à tout construire aujourd'hui. Tu dois savoir que ça existe, pour ne pas crier victoire trop tôt après un score de validation. Prévois dès le jour 1 un chemin "humain si doute".

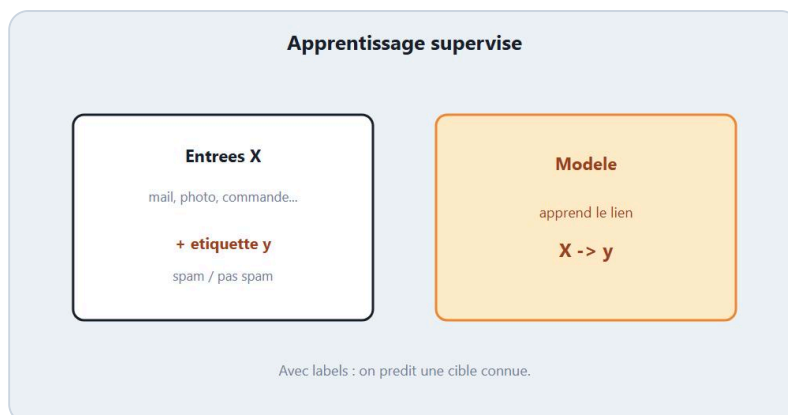
Culture et communication

Apprends à dire "non" à un modèle inutile. Apprends à dire "pas encore" quand les labels manquent. Apprends à dire "voici les limites" quand tu présentes un score. Cette honnêteté te rend plus crédible que n'importe quel jargon. Chez DanielCraft, on forme des gens capables de tenir cette conversation avec un métier, un manager, ou un client.

Pour aller plus loin sans te perdre

Tu n'as pas à tout maîtriser d'un coup. Reviens à ce chapitre quand tu butes sur un cas réel. Relis l'erreur classique. Refais le "A toi". Chez DanielCraft, on préfère trois relectures actives à une lecture passive de vingt pages. Si un paragraphe te semble encore flou, reformule-le à voix haute avec ton propre exemple. Dès que ça passe à l'oral, c'est que c'est entre. Ensuite seulement, passe au chapitre suivant. Cette discipline lente crée une compétence rapide sur la durée - le contraire du binge de tutos oubliés le lendemain.

Chapitre 2 - Apprentissage supervise : apprendre avec les bonnes reponses



Supervise : etiquettes + predictions.

L'apprentissage **supervise**, c'est quand chaque exemple d'entrainement vient avec la "bonne reponse". On appelle souvent X les entrees (**features**) et y la cible (**label**). Exemple : surface et quartier (X) -> prix du loyer (y). Ou : texte du mail (X) -> spam / pas spam (y). Le modele apprend une relation. Ensuite, sur de nouveaux X, il propose un y.

Pourquoi "supervise" ? Parce que les labels jouent le role d'un superviseur : ils corrigent. Sans labels, tu ne peux pas faire de supervise classique. Or labelliser coute cher (temps humain, expertise). C'est souvent le vrai frein des projets, pas l'algorithme a la mode.

★ A RETENIR

Sans labels clairs, pas de supervise. La definition de y est un cahier des charges, pas un detail.

Deux grandes familles

Si y est un nombre continu (prix, temperature, duree), on parle souvent de **regression**. Si y est une categorie (spam/ham, malade/sain, chat/chien, risque faible/moyen/eleve), on parle de **classification**. Les frontieres peuvent etre floues (une note sur 5 peut etre traitee comme nombre ou comme classes). L'important est de savoir ce que tu veux predire et comment tu mesureras l'erreur.

Le deroulement mental

1) Collecter des exemples labels. 2) Nettoyer. 3) Separer **train/test**. 4) Choisir un modele simple. 5) Entrainer sur train. 6) Evaluer sur test. 7) Inspecter les erreurs. 8) Iterer sur les features et les donnees avant de complexifier le modele. Chez DanielCraft, on insiste : les donnees et la question metier d'abord, la usine a gaz algo ensuite.

♦ ASTUCE

Avant de choisir un algo, ecris en une phrase : "je predit Y a partir de X pour decider Z". Si Z est flou, pause.

Exemple Noe

Noe labelise 2 000 commandes : retour oui/non. Features possibles : montant, categorie produit, delai de livraison promis, client deja revenu ou non, pays. Cible : retour. C'est du supervise (classification). S'il veut predire le montant du prochain panier, c'est une regression. Meme boutique, deux questions, deux cibles.

Limites

Si tes labels sont sales (annotateurs fatigues, definitions floues), le modele apprend le flou. Si tes exemples ne representent pas le futur (tu n'as que des clients urbains et tu deploies a la campagne), le modele generalise mal. Le supervise n'est pas une garantie de justice ni de verite ; c'est une methode d'apprentissage a partir d'exemples.

Erreur classique

Entrainer et "tester" sur les memes lignes, puis crier victoire. Ou changer la definition du label en cours de route sans le dire. Clarifie y comme tu clarifierais un cahier des charges.

⚠ ATTENTION

Tester sur les memes lignes que l'entrainement, c'est se mentir. Split d'abord, score ensuite.

A toi

Decris un jeu supervise perso : que sont X, que sont y, qui labellise, combien d'exemples tu pourrais avoir en un mois.

Qualite des labels

Un label flou produit un modele flou. Si deux annotateurs ne sont pas d'accord sur "retour abusif" versus "retour legitime", le modele apprend le desaccord. Investis du temps dans une definition ecrite, des exemples frontieres, et parfois un double annotation sur un echantillon. C'est moins glamorous que d'essayer un nouvel algo. C'est souvent ce qui decide du succes.

Volume : combien faut-il ?

Ca depend de la difficulte et du nombre de features. Des centaines peuvent suffire pour une separation simple. Des dizaines de milliers pour un probleme bruyant. Des millions dans certains domaines. Plutot que de mythifier un chiffre, commence petit, mesure, regarde les erreurs, decide si plus de labels changerait vraiment la donne. Parfois oui. Parfois le probleme est mal pose.

Developpement : penser comme un artisan des modeles

Le machine learning n'est pas un distributeur de verite. C'est un artisanat de decisions sous incertitude. Tu choisis une question, tu rassembles des exemples, tu acceptes une erreur moyenne, tu te donnes les moyens de la mesurer, tu decides si cette erreur est tolerable pour le cas d'usage. Beaucoup de frustration vient d'attendre la perfection la ou il fallait un score utile avec un humain dans la boucle.

Quand Noe predit un risque de retour, il ne remplace pas le service client. Il priorise. Quand un hopital utilise un score (hors du perimetre de ce livre introductif, et avec des cadres stricts), l'enjeu n'est plus le meme : les couts d'erreur explosent, les biais deviennent critiques, la gouvernance monte. Adapte toujours la profondeur de ta demarche a l'impact. Un modele jouet sur un CSV public n'exige pas la meme revue qu'un score qui bloque un credit.

Donnees : le personnage principal

Les algorithmes changent. Les principes de donnees restent : definition claire, representativite, fraicheur, droits, documentation, absence de fuite, inspection des cas bizarres. Passe plus de temps sur les donnees que sur le shopping d'algorithmes. C'est le conseil le moins glamour et le plus rentable du livre. Un arbre simple sur des features excellentes bat une usine a gaz sur un tableau sale.

Mise en production (aperçu)

Un notebook n'est pas un produit. En production, tu dois gerer des entrees manquantes nouvelles, des categories inconnues, des delais, des journaux, des versions, des rollback, des alertes si la metrique chute. Tu n'as pas a tout construire aujourd'hui. Tu dois savoir que ca existe, pour ne pas crier victoire trop tot apres un score de validation. Prevour des le jour 1 un chemin "humain si doute".

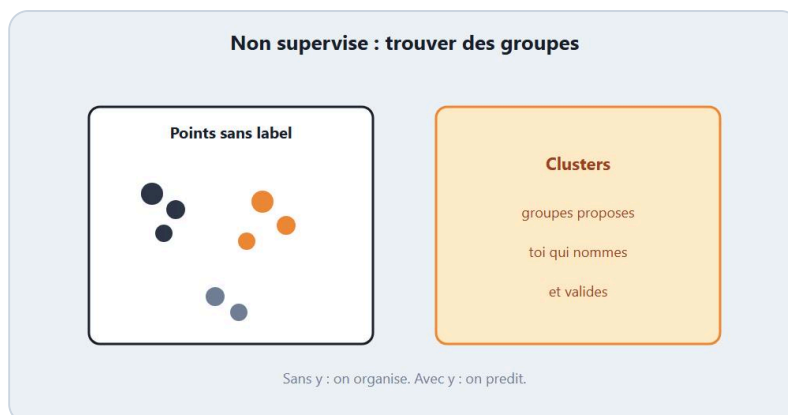
Culture et communication

Apprends a dire "non" a un modele inutile. Apprends a dire "pas encore" quand les labels manquent. Apprends a dire "voici les limites" quand tu presentes un score. Cette honnetete te rend plus credible que n'importe quel jargon. Chez DanielCraft, on forme des gens capables de tenir cette conversation avec un metier, un manager, ou un client.

Pour aller plus loin sans te perdre

Tu n'as pas a tout maitriser d'un coup. Reviens a ce chapitre quand tu butes sur un cas reel. Relis l'erreur classique. Refais le "A toi". Chez DanielCraft, on prefere trois relectures actives a une lecture passive de vingt pages. Si un paragraphe te semble encore flou, reformule-le a voix haute avec ton propre exemple. Des que ca passe a l'oral, c'est que c'est entre. Ensuite seulement, passe au chapitre suivant. Cette discipline lente cree une competence rapide sur la duree - le contraire du binge de tutos oublies le lendemain.

Chapitre 3 - Apprentissage non supervise : trouver des structures sans label



Non supervise : regrouper sans label.

Parfois, tu n'as pas de bonne reponse y. Tu as seulement des descriptions de clients, de produits, de comportements. L'apprentissage **non supervise** cherche des structures : des groupes (**clustering**), des reductions de dimensions, des motifs inhabituels. Personne n'a dit "voici la verite" ; le modele propose une organisation. A toi de juger si elle est utile.

★ A RETENIR

Sans label, on organise. Avec label, on predict une cible. Ne confonds pas les deux outils.

Clustering : regrouper

L'idée : mettre ensemble ce qui se ressemble selon certaines mesures. Noe peut regrouper ses clients en "acheteurs occasionnels petit panier", "fideles gros panier", "chasseurs de promo". Ces noms, c'est lui qui les donne apres coup. L'algorithme sort des groupes chiffres ; l'humain interprete. Si l'interpretation est forcee, le clustering devient de la science fiction marketing.

Quand s'en servir

Explorer. Segmenter pour adapter un message. Detecter des anomalies (un point tres loin des autres). Reduire la complexite avant une autre etape. Ce n'est generalement pas le meilleur outil si tu as deja un **label** clair et une decision binaire a prendre - dans ce cas, le **supervise** est plus direct.

♦ ASTUCE

Avant de fixer le nombre de clusters, demande : "quelle action change si ce groupe existe ?" Si aucune, tu explores encore.

Attention aux illusions

Les groupes dependent des **features** choisies et de l'echelle des variables. Si tu melanges "age" et "revenu annuel" sans normaliser, une variable ecrase l'autre. Si tu demandes 8 clusters parce que "ca fait joli", tu obtiendras 8 clusters meme si 3 suffisaient. Le non supervise demande de la sobriete et de la validation metier : est-ce que ces groupes changent une action ?

⚠ ATTENTION

Normalise (ou standardise) avant de clusterer des variables d'echelles differentes. Sinon une seule colonne dicte les groupes.

Lien avec le reste de l'IA

Les **embeddings** (representations vectorielles) utilises autour des LLM sont cousins de cette logique : mettre proche ce qui "se ressemble" dans un espace. Tu n'as pas besoin des maths ici. Retiens l'intuition : sans label, on organise ; avec label, on predit une cible.

Erreur classique

Prendre les clusters comme une verite sociologique. Ou les vendre a un client comme "preuves scientifiques" sans test terrain. Autre piege : oublier la saisonnalite (un cluster "inactif" en aout n'est pas un cluster "perdu").

A toi

Liste 10 entites de ton monde (clients, tickets, articles). Quelles features utiliserais-tu pour les regrouper ? Quelle decision changerait si les groupes etaient bons ?

Valider un clustering sans label magique

Tu peux regarder la stabilite (est-ce que les groupes bougent beaucoup si tu reechantillones ?). Tu peux demander a un expert metier si les groupes suggerent des actions differentes. Tu peux mesurer un indicateur externe (taux de reponse a une campagne par cluster). Sans au moins une de ces validations, le clustering reste une jolie projection, pas une decision.

Developpement : penser comme un artisan des modeles

Le machine learning n'est pas un distributeur de verite. C'est un artisanat de decisions sous incertitude. Tu choisis une question, tu rassembles des exemples, tu acceptes une erreur moyenne, tu te donnes les moyens de la mesurer, tu decides si cette erreur est tolerable pour le cas d'usage. Beaucoup de frustration vient d'attendre la perfection la ou il fallait un score utile avec un humain dans la boucle.

Quand Noe predit un risque de retour, il ne remplace pas le service client. Il priorise. Quand un hopital utilise un score (hors du perimetre de ce livre introductif, et avec des cadres stricts), l'enjeu n'est plus le meme : les couts d'erreur explosent, les biais deviennent critiques, la gouvernance monte. Adapte toujours la profondeur de ta demarche a l'impact. Un modele jouet sur un CSV public n'exige pas la meme revue qu'un score qui bloque un credit.

Donnees : le personnage principal

Les algorithmes changent. Les principes de donnees restent : definition claire, representativite, fraicheur, droits, documentation, absence de fuite, inspection des cas bizarres. Passe plus de temps sur les donnees que sur le shopping d'algorithmes. C'est le conseil le moins glamour et le plus rentable du livre. Un arbre simple sur des features excellentes bat une usine a gaz sur un tableau sale.

Mise en production (aperçu)

Un notebook n'est pas un produit. En production, tu dois gérer des entrées manquantes nouvelles, des catégories inconnues, des délais, des journaux, des versions, des rollback, des alertes si la métrique chute. Tu n'as pas à tout construire aujourd'hui. Tu dois savoir que ça existe, pour ne pas crier victoire trop tôt après un score de validation. Prévois dès le jour 1 un chemin "humain si doute".

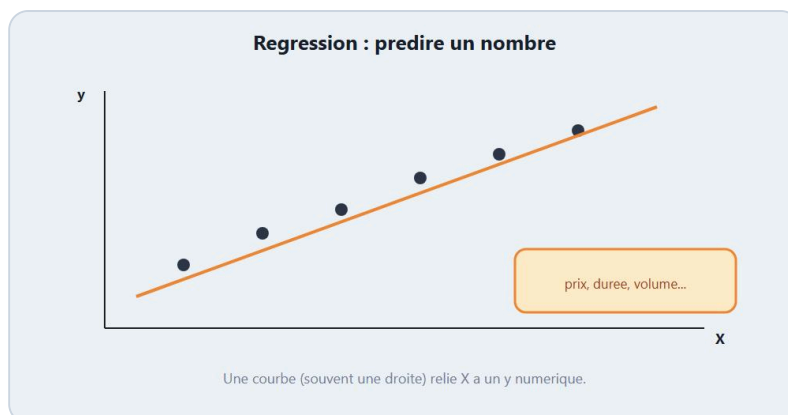
Culture et communication

Apprends à dire "non" à un modèle inutile. Apprends à dire "pas encore" quand les labels manquent. Apprends à dire "voici les limites" quand tu présentes un score. Cette honnêteté te rend plus crédible que n'importe quel jargon. Chez DanielCraft, on forme des gens capables de tenir cette conversation avec un métier, un manager, ou un client.

Pour aller plus loin sans te perdre

Tu n'as pas à tout maîtriser d'un coup. Reviens à ce chapitre quand tu butes sur un cas réel. Relis l'erreur classique. Refais le "A toi". Chez DanielCraft, on préfère trois relectures actives à une lecture passive de vingt pages. Si un paragraphe te semble encore flou, reformule-le à voix haute avec ton propre exemple. Dès que ça passe à l'oral, c'est que c'est entre. Ensuite seulement, passe au chapitre suivant. Cette discipline lente crée une compétence rapide sur la durée - le contraire du binge de tutos oubliés le lendemain.

Chapitre 4 - Regression : predire un nombre



Regression : predire un nombre.

La **régression** vise à prédire une valeur numérique. Prix, durée, température, nombre de visites, panier moyen. L'idée la plus simple : une courbe (souvent une droite au début) qui relie des X à un y. Tu ajustes la courbe pour qu'elle passe "au mieux" près des points d'entraînement, puis tu lis la prédiction pour un nouveau X.

★ A RETENIR

Regression = prédire un nombre. Commence simple, mesure, complexifie seulement si le gain est réel.

Regression lineaire : l'intuition

Imagine prédire le loyer à partir de la surface. En gros, $\text{loyer} \approx a * \text{surface} + b$. Le modèle cherche a et b qui minimisent une erreur (souvent la distance verticale aux points). Ce n'est pas toujours réaliste (les relations sont courbes, il y a d'autres facteurs), mais c'est un excellent premier modèle : simple, interpretable, rapide à diagnostiquer.

Au-delà de la droite

Tu peux ajouter des **features** (quartier, étage, présence d'ascenseur). Tu peux transformer (log du prix). Tu peux utiliser des modèles plus souples (arbres, forêts). La règle DanielCraft : commence simple, mesure, complexifie seulement si le gain est réel sur le jeu de **test** et en conditions proches du déploiement.

♦ ASTUCE

Garde toujours une **baseline** "prédire la moyenne". Si ta régression ne la bat pas franchement, le problème est ailleurs (données, question, fuites).

Erreurs et unités

Une erreur de 50 euros sur un loyer à 500 euros n'est pas la même chose qu'une erreur de 50 euros sur un loyer à 3 000 euros. D'où l'importance des **metriques** (chapitre dédié) et du sens métier. Noé qui prédit des ventes veut savoir s'il se trompe de 5 unités ou de 500 - ça change les commandes fournisseurs.

Pieges

Fuites de donnees (utiliser une info du futur). Outliers qui tirent la droite. Relations non lineaires ignorees. Croire que R^2 eleve sur train suffit. Predire un nombre puis le traiter comme une certitude dans une decision automatique sans marge.

⚠ ATTENTION

N'extrapolate pas hors de la zone vue a l'entrainement. Une droite sur 20-80 m² ne "sait" pas predire 300 m².

Erreur classique

Extrapolateur fou : entrainer sur des surfaces de 20 a 80 m² puis predire pour 300 m² comme si la droite restait valable. Les modeles sont souvent locaux a la zone vue.

A toi

Choisis une cible numerique. Dessine a la main une relation simple avec une feature. Ou croit-elle ? Ou casse-t-elle ?

Interpretable d'abord

Une **regression lineaire** bien faite te dit : "toutes choses egales par ailleurs dans le modele, +10 m², +X euros". Attention au piege causal : ce n'est pas une preuve que agrandir cree le loyer. Mais pour diagnostiquer et communiquer, c'est precieux. Les modeles boites noires viendront plus tard, si le gain le justifie.

Developpement : penser comme un artisan des modeles

Le machine learning n'est pas un distributeur de verite. C'est un artisanat de decisions sous incertitude. Tu choisis une question, tu rassembles des exemples, tu acceptes une erreur moyenne, tu te donnes les moyens de la mesurer, tu decides si cette erreur est tolerable pour le cas d'usage. Beaucoup de frustration vient d'attendre la perfection la ou il fallait un score utile avec un humain dans la boucle.

Quand Noe predit un risque de retour, il ne remplace pas le service client. Il priorise. Quand un hopital utilise un score (hors du perimetre de ce livre introductif, et avec des cadres stricts), l'enjeu n'est plus le meme : les couts d'erreur explosent, les biais deviennent critiques, la gouvernance monte. Adapte toujours la profondeur de ta demarche a l'impact. Un modele jouet sur un CSV public n'exige pas la meme revue qu'un score qui bloque un credit.

Donnees : le personnage principal

Les algorithmes changent. Les principes de donnees restent : definition claire, representativite, fraicheur, droits, documentation, absence de fuite, inspection des cas bizarres. Passe plus de temps sur les donnees que sur le shopping d'algorithmes. C'est le conseil le moins glamour et le plus rentable du livre. Un arbre simple sur des features excellentes bat une usine a gaz sur un tableau sale.

Mise en production (aperçu)

Un notebook n'est pas un produit. En production, tu dois gérer des entrées manquantes nouvelles, des catégories inconnues, des délais, des journaux, des versions, des rollback, des alertes si la métrique chute. Tu n'as pas à tout construire aujourd'hui. Tu dois savoir que ça existe, pour ne pas crier victoire trop tôt après un score de validation. Prévois dès le jour 1 un chemin "humain si doute".

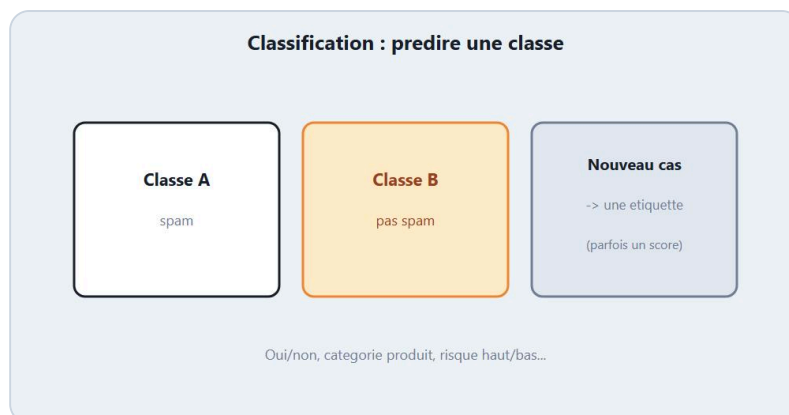
Culture et communication

Apprends à dire "non" à un modèle inutile. Apprends à dire "pas encore" quand les labels manquent. Apprends à dire "voici les limites" quand tu présentes un score. Cette honnêteté te rend plus crédible que n'importe quel jargon. Chez DanielCraft, on forme des gens capables de tenir cette conversation avec un métier, un manager, ou un client.

Pour aller plus loin sans te perdre

Tu n'as pas à tout maîtriser d'un coup. Reviens à ce chapitre quand tu butes sur un cas réel. Relis l'erreur classique. Refais le "A toi". Chez DanielCraft, on préfère trois relectures actives à une lecture passive de vingt pages. Si un paragraphe te semble encore flou, reformule-le à voix haute avec ton propre exemple. Dès que ça passe à l'oral, c'est que c'est entre. Ensuite seulement, passe au chapitre suivant. Cette discipline lente crée une compétence rapide sur la durée - le contraire du binge de tutos oubliés le lendemain.

Chapitre 5 - Classification : predire une classe



Classification : predire une classe.

Classifier, c'est attribuer une categorie. Spam ou non. Client a risque ou non. Produit defaillant ou non. Langue detectee. Type de ticket support. La sortie n'est pas un prix ; c'est une case (parfois un score de probabilite avant la case).

★ A RETENIR

Classification = predire une classe. Le seuil et le cout des erreurs comptent plus qu'un score "joli".

Frontiere de decision

Imagine des points de deux couleurs sur un plan. Un classifieur trace une frontiere. D'un cote, classe A ; de l'autre, classe B. Lineaire : une droite. Plus souple : des courbes, des rectangles (arbres), des combines. Sur des textes ou des images, l'espace est vaste, mais l'idee reste : separer des regions.

Scores et seuils

Beaucoup de modeles sortent un score ("probabilite" de spam). Tu choisis un **seuil** : au-dessus, tu filtres. Baisser le seuil attrape plus de spam mais risque de bloquer des mails legimes. Monter le seuil l'inverse. Le "meilleur" seuil depend du cout des erreurs - pas d'une beaute mathematique.

♦ ASTUCE

Ecris d'abord le cout d'un **faux positif** et d'un **faux negatif**. Le seuil se deduit souvent de ca, pas d'une courbe seule.

Classes desequilibrees

Si 1 % des commandes sont frauduleuses, un modele naif qui dit toujours "pas fraude" a 99 % de exactitude globale... et zero interet. Il faut des **metriques** adaptees (precision, rappel, F1, cout metier) et parfois des strategies d'echantillonnage. Noe avec 8 % de retours doit regarder la detection des retours, pas seulement le taux global de bonnes reponses.

⚠ ATTENTION

Sur classes desequilibrees, l'**accuracy** globale peut etre excellente et le modele inutile. Regarde precision, rappel, et la matrice de confusion.

Multiclasse

Parfois plus de deux cases : type A/B/C/D. Les idees restent proches, les metriques et matrices de confusion s'etendent. Clarifie si les classes sont ordonnees (faible/moyen/fort) ou non : ca change les options.

Erreur classique

Optimiser l'accuracy aveugle sur un jeu desequilibre. Ou deployer un classifieur sans definir le cout d'un faux positif vs faux negatif (bloquer un client sain vs laisser passer une fraude).

A toi

Pour une classification de ton monde, ecris : cout d'un faux positif, cout d'un faux negatif. Lequel est pire ? Ca guidera ton seuil.

Probabilites calibrees (idee)

Un score de 0,8 devrait vouloir dire "environ 80 % de chances" si tu veux l'utiliser comme probabilite. Ce n'est pas toujours vrai hors boite. Pour debuter, traite les scores comme des ordres utiles pour ranger, choisis un seuil metier, et mefie-toi des interpretations trop litterales tant que tu n'as pas verifie la calibration.

Developpement : penser comme un artisan des modeles

Le machine learning n'est pas un distributeur de verite. C'est un artisanat de decisions sous incertitude. Tu choisis une question, tu rassembles des exemples, tu acceptes une erreur moyenne, tu te donnes les moyens de la mesurer, tu decides si cette erreur est tolerable pour le cas d'usage. Beaucoup de frustration vient d'attendre la perfection la ou il fallait un score utile avec un humain dans la boucle.

Quand Noe predit un risque de retour, il ne remplace pas le service client. Il priorise. Quand un hopital utilise un score (hors du perimetre de ce livre introductif, et avec des cadres stricts), l'enjeu n'est plus le meme : les couts d'erreur explosent, les biais deviennent critiques, la gouvernance monte. Adapte toujours la profondeur de ta demarche a l'impact. Un modele jouet sur un CSV public n'exige pas la meme revue qu'un score qui bloque un credit.

Donnees : le personnage principal

Les algorithmes changent. Les principes de donnees restent : definition claire, representativite, fraicheur, droits, documentation, absence de fuite, inspection des cas bizarres. Passe plus de temps sur les donnees que sur le shopping d'algorithmes. C'est le conseil le moins glamour et le plus rentable du livre. Un arbre simple sur des features excellentes bat une usine a gaz sur un tableau sale.

Mise en production (aperçu)

Un notebook n'est pas un produit. En production, tu dois gérer des entrées manquantes nouvelles, des catégories inconnues, des délais, des journaux, des versions, des rollback, des alertes si la métrique chute. Tu n'as pas à tout construire aujourd'hui. Tu dois savoir que ça existe, pour ne pas crier victoire trop tôt après un score de validation. Prévois dès le jour 1 un chemin "humain si doute".

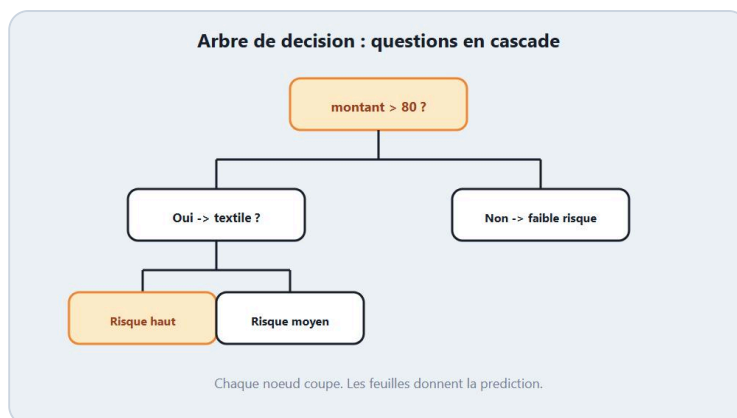
Culture et communication

Apprends à dire "non" à un modèle inutile. Apprends à dire "pas encore" quand les labels manquent. Apprends à dire "voici les limites" quand tu présentes un score. Cette honnêteté te rend plus crédible que n'importe quel jargon. Chez DanielCraft, on forme des gens capables de tenir cette conversation avec un métier, un manager, ou un client.

Pour aller plus loin sans te perdre

Tu n'as pas à tout maîtriser d'un coup. Reviens à ce chapitre quand tu butes sur un cas réel. Relis l'erreur classique. Refais le "A toi". Chez DanielCraft, on préfère trois relectures actives à une lecture passive de vingt pages. Si un paragraphe te semble encore flou, reformule-le à voix haute avec ton propre exemple. Dès que ça passe à l'oral, c'est que c'est entre. Ensuite seulement, passe au chapitre suivant. Cette discipline lente crée une compétence rapide sur la durée - le contraire du binge de tutos oubliés le lendemain.

Chapitre 6 - Arbres de decision : des questions en cascade



Arbre : questions en cascade jusqu'a une feuille.

Un **arbre de decision**, c'est un enchainement de questions simples sur les **features**, jusqu'a une feuille qui donne une prediction. Exemple : montant > 100 ? oui -> client deja revenu ? non -> risque de retour eleve. C'est lisible. Ca plait aux metiers. Ca capture des interactions (tel effet seulement si telle condition).

★ A RETENIR

Un arbre pose des questions en cascade. Lisible pour le metier, fragile s'il devient trop profond.

Pourquoi c'est pedagogique

Tu peux montrer l'arbre a Noe et il comprend sans equation. Tu vois ou le modele coupe. Tu detectes des regles absurdes ("si code postal = 99999 alors ...") qui revelent un souci de donnees. Pour enseigner le machine learning, les arbres sont un pont en or entre intuition et pratique.

Limites d'un arbre seul

Un arbre profond colle trop aux donnees d'entrainement (**overfitting**). Un arbre trop petit sous-apprend. Les frontieres sont souvent paralleles aux axes (questions du type feature > seuil), ce qui peut etre maladroit pour certaines formes. La solution courante : des ensembles d'arbres (**forets**, boosting). L'idée : plusieurs arbres votent ou se corrigent, plus robustes, un peu moins lisibles.

⚠ ATTENTION

Un arbre geant n'est pas un trophée. C'est souvent un drapeau rouge d'overfitting. Limite la profondeur et verifie sur **test**.

Regles de bonne vie

Commence par un arbre peu profond pour explorer. Controle la profondeur. Verifie sur test. Regarde l'importance des features (quelles questions servent souvent). Ne prends pas une importance comme une preuve causale : "associe a" n'est pas "cause".

♦ ASTUCE

Interdis les IDs uniques (numero de commande, email) comme features. L'arbre s'en sert comme antiseche, puis echoue en production.

Erreur classique

Montrer un arbre geant comme preuve de sophistication. Un arbre geant est souvent un drapeau rouge d'overfitting. Autre piege : laisser des IDs uniques (numero de commande) comme feature : l'arbre s'en sert comme antiseche, puis echoue en production.

A toi

Sans code, ecris un mini-arbre a 3 questions pour une decision de ton quotidien pro. Est-il juste ? Ou revele-t-il un biais ?

Forets et boosting (aperçu)

Une **foret aleatoire** : beaucoup d'arbres, chacun voit un sous-ensemble, vote. Plus robuste, moins interpretable qu'un seul arbre. Le boosting : des arbres qui corrigent les erreurs les uns des autres, souvent tres performants sur tableaux. Tu peux les utiliser plus tard ; comprends d'abord un arbre seul pour garder l'intuition.

Developpement : penser comme un artisan des modeles

Le machine learning n'est pas un distributeur de verite. C'est un artisanat de decisions sous incertitude. Tu choisis une question, tu rassembles des exemples, tu acceptes une erreur moyenne, tu te donnes les moyens de la mesurer, tu decides si cette erreur est tolerable pour le cas d'usage. Beaucoup de frustration vient d'attendre la perfection la ou il fallait un score utile avec un humain dans la boucle.

Quand Noe predit un risque de retour, il ne remplace pas le service client. Il priorise. Quand un hopital utilise un score (hors du perimetre de ce livre introductif, et avec des cadres stricts), l'enjeu n'est plus le meme : les couts d'erreur explosent, les biais deviennent critiques, la gouvernance monte. Adapte toujours la profondeur de ta demarche a l'impact. Un modele jouet sur un CSV public n'exige pas la meme revue qu'un score qui bloque un credit.

Donnees : le personnage principal

Les algorithmes changent. Les principes de donnees restent : definition claire, representativite, fraicheur, droits, documentation, absence de fuite, inspection des cas bizarres. Passe plus de temps sur les donnees que sur le shopping d'algorithmes. C'est le conseil le moins glamour et le plus rentable du livre. Un arbre simple sur des features excellentes bat une usine a gaz sur un tableau sale.

Mise en production (aperçu)

Un notebook n'est pas un produit. En production, tu dois gerer des entrees manquantes nouvelles, des categories inconnues, des delais, des journaux, des versions, des rollback, des alertes si la metrique chute. Tu n'as pas a tout construire aujourd'hui. Tu dois savoir que ca existe, pour ne pas crier victoire trop tot apres un score de validation. Prevour des le jour 1 un chemin "humain si doute".

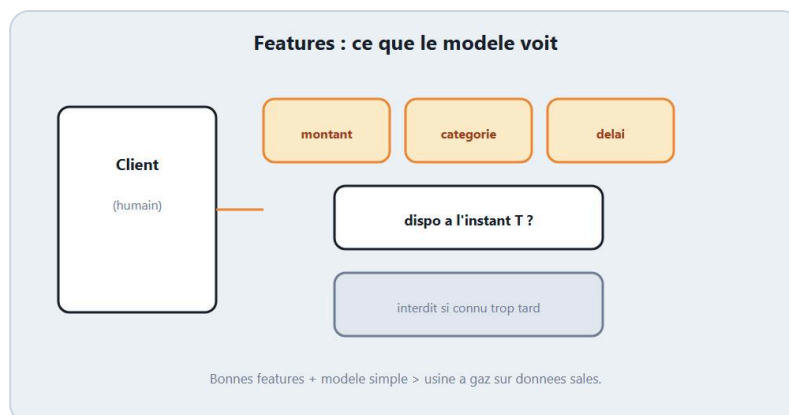
Culture et communication

Apprends a dire "non" a un modele inutile. Apprends a dire "pas encore" quand les labels manquent. Apprends a dire "voici les limites" quand tu presentes un score. Cette honnetete te rend plus credible que n'importe quel jargon. Chez DanielCraft, on forme des gens capables de tenir cette conversation avec un metier, un manager, ou un client.

Pour aller plus loin sans te perdre

Tu n'as pas a tout maitriser d'un coup. Reviens a ce chapitre quand tu butes sur un cas reel. Relis l'erreur classique. Refais le "A toi". Chez DanielCraft, on prefere trois relectures actives a une lecture passive de vingt pages. Si un paragraphe te semble encore flou, reformule-le a voix haute avec ton propre exemple. Des que ca passe a l'oral, c'est que c'est entre. Ensuite seulement, passe au chapitre suivant. Cette discipline lente cree une competence rapide sur la duree - le contraire du binge de tutos oublies le lendemain.

Chapitre 7 - Features : ce que le modele "voit"



Features : ce que le modele voit, dispo a l'instant T.

Les **features** sont les caracteristiques en entree. Le modele ne voit pas "un client". Il voit des nombres, des categories encodees, des indicateurs. Si tu lui donnes de mauvaises features, le plus bel algorithme restera myope. A l'inverse, de bonnes features avec un modele simple battent souvent un modele complexe sur des entrees pauvres.

★ A RETENIR

Le modele ne voit que tes features. Qualite et disponibilite a l'instant T battent la complexite de l'algo.

Types courants

Numeriques (age, montant). Categoriques (pays, categorie produit). Boolennes (deja client). Temporelles (jour de la semaine, mois - attention aux fuites). Texte (qui demande un travail de transformation). Issues de calcul (ratio panier / moyenne historique). Chez DanielCraft, on encourage a ecrire la liste des features comme un contrat : nom, definition, comment calculee, disponible au moment de la prediction ?

Disponible au bon moment

Si tu predit un retour au moment de l'achat, tu n'as pas le droit d'utiliser "a contacte le support 3 jours apres". Ca n'existe pas encore. Cette **fuite** temporelle rend les scores magnifiques en labo et nuls en vrai. Pose toujours la question : est-ce que j'aurai cette info a l'instant T ou je predit ?

⚠ ATTENTION

Toute info connue seulement apres la prediction est interdite. Sinon ton score labo ment.

Nettoyage et preparation

Valeurs manquantes : imputer, signaler, ou exclure selon le cas. Unites coherentes. Outliers inspectes. Categories rares regroupees. Texte normalise si besoin. Scaling pour certains modeles (pas toujours pour les arbres). Le **pipeline** (chapitre dedie) industrialise ces gestes pour ne pas tricher entre train et test.

Feature engineering

C'est l'art de créer des variables utiles : jour ferie oui/non, distance a un entrepot, prix relatif a la categorie, nombre d'achats 90 derniers jours. Ca demande du sens metier. Noe sait que les retours explosent sur une taille de textile ; une feature "categorie = textile" + "taille commandee vs taille habituelle" peut valoir mieux qu'un reseau de neurones mal nourri.

♦ ASTUCE

Noe cree "delai promis - delai median historique" plutot que le seul delai brut. Une feature relative bat souvent une colonne brute.

Erreur classique

Tout garder "au cas ou" (bruit, fuites, colonnes identifiantes). Ou tout jeter trop vite. Autre piege : encoder mal une categorie ordinaire comme un nombre (Paris=1, Lyon=2, Marseille=3) et laisser un modele lineaire croire a un ordre absurde.

A toi

Pour ton sujet, liste 8 features candidates. Barre celles indisponibles a l'instant de prediction. Il reste quoi ?

Documentation des features

Nom, definition, formule, source, fraicheur, dispo a T, valeurs manquantes possibles, exemples. Une page tableur suffit au debut. Quand le projet grossit, cette page evite les conflits ("moi je croyais que montant = TTC"). Chez DanielCraft, on dit qu'une feature non documentee est une dette.

Developpement : penser comme un artisan des modeles

Le machine learning n'est pas un distributeur de verite. C'est un artisanat de decisions sous incertitude. Tu choisis une question, tu rassembles des exemples, tu acceptes une erreur moyenne, tu te donnes les moyens de la mesurer, tu decides si cette erreur est tolerable pour le cas d'usage. Beaucoup de frustration vient d'attendre la perfection la ou il fallait un score utile avec un humain dans la boucle.

Quand Noe predit un risque de retour, il ne remplace pas le service client. Il priorise. Quand un hopital utilise un score (hors du perimetre de ce livre introductif, et avec des cadres stricts), l'enjeu n'est plus le meme : les couts d'erreur explosent, les biais deviennent critiques, la gouvernance monte. Adapte toujours la profondeur de ta demarche a l'impact. Un modele jouet sur un CSV public n'exige pas la meme revue qu'un score qui bloque un credit.

Donnees : le personnage principal

Les algorithmes changent. Les principes de donnees restent : definition claire, representativite, fraicheur, droits, documentation, absence de fuite, inspection des cas bizarres. Passe plus de temps sur les donnees que sur le shopping d'algorithmes. C'est le conseil le moins glamour et le plus rentable du livre. Un arbre simple sur des features excellentes bat une usine a gaz sur un tableau sale.

Mise en production (aperçu)

Un notebook n'est pas un produit. En production, tu dois gérer des entrées manquantes nouvelles, des catégories inconnues, des délais, des journaux, des versions, des rollback, des alertes si la métrique chute. Tu n'as pas à tout construire aujourd'hui. Tu dois savoir que ça existe, pour ne pas crier victoire trop tôt après un score de validation. Prévois dès le jour 1 un chemin "humain si doute".

Culture et communication

Apprends à dire "non" à un modèle inutile. Apprends à dire "pas encore" quand les labels manquent. Apprends à dire "voici les limites" quand tu présentes un score. Cette honnêteté te rend plus crédible que n'importe quel jargon. Chez DanielCraft, on forme des gens capables de tenir cette conversation avec un métier, un manager, ou un client.

Pour aller plus loin sans te perdre

Tu n'as pas à tout maîtriser d'un coup. Reviens à ce chapitre quand tu butes sur un cas réel. Relis l'erreur classique. Refais le "A toi". Chez DanielCraft, on préfère trois relectures actives à une lecture passive de vingt pages. Si un paragraphe te semble encore flou, reformule-le à voix haute avec ton propre exemple. Dès que ça passe à l'oral, c'est que c'est entre. Ensuite seulement, passe au chapitre suivant. Cette discipline lente crée une compétence rapide sur la durée - le contraire du binge de tutos oubliés le lendemain.

Chapitre 8 - Train / test : apprendre sans se mentir



Train pour apprendre, test pour evaluer.

Si tu evalues un modele sur les memes exemples que ceux qui ont servi a l'entrainer, tu te mentiras presque toujours : le modele a pu "retenir" au lieu de generaliser. D'ou le geste sacre : separer les donnees. **Train** pour apprendre. **Test** pour juger, une fois, proprement. Parfois un jeu de **validation** pour choisir des reglages, puis un test final touche rarement.

★ A RETENIR

Train pour apprendre, test pour juger. Sans split honnete, tout score est suspect.

Comment separer

Aleatoire quand les lignes sont independantes. Temporelle quand le temps compte (entraine sur le passe, teste sur le futur) - souvent plus realiste pour Noe et les ventes. Par groupe (par client) si plusieurs lignes appartiennent a la meme entite et que tu ne veux pas de fuite. Le pourcentage classique 80/20 n'est qu'une habitude ; l'important est le protocole honnete.

♦ ASTUCE

Si ton monde change dans le temps (ventes, saison), prefere un split temporel a un tirage aleatoire. C'est plus proche de la prod.

Validation croisee (idee)

Quand tu as peu de donnees, tu peux tourner plusieurs decoupes (**cross-validation**) et moyenner les scores. Ca stabilise l'estimation. Ca ne remplace pas un vrai test sur des donnees plus recentes si ton monde change.

Fuite (data leakage)

Toute information du test qui contamine le train ou le preprocessing fait exploser les scores artificiellement. Exemples : normaliser sur tout le dataset avant split ; choisir les features en regardant le test ; inclure la cible deguisee. Le **pipeline** doit ajuster ses transformations sur le train uniquement, puis les appliquer au test.

⚠ ATTENTION

Normaliser ou encoder sur tout le CSV avant le split, c'est de la fuite. Fit sur le train seulement.

Ritual DanielCraft

1) Définir la question et le moment de prediction. 2) Split honnête. 3) **Baseline** stupide (toujours prédire la moyenne / la classe majoritaire). 4) Modèle simple. 5) Comparer au baseline sur test. Si tu ne bats pas le baseline, tu n'as pas encore de victoire.

Erreur classique

Retoucher le modèle jusqu'à ce que le score test soit parfait... en regardant le test cinquante fois. Tu as transformé le test en train mental. Garde un jeu final rare, ou accepte une validation croisée déclarée.

A toi

Decris ton split : méthode (aléatoire / temps / groupe), proportions, baseline stupide. Ecris pourquoi ce split évite (ou non) la fuite.

Simulation de production

Le meilleur test ressemble au futur : mêmes sources, mêmes délais, mêmes manquants. Si en labo tu as des champs complets et en prod la moitié manquent, ton score labo ment. Construis un jeu qui imite la prod, même plus petit.

Developpement : penser comme un artisan des modèles

Le machine learning n'est pas un distributeur de vérité. C'est un artisanat de décisions sous incertitude. Tu choisis une question, tu rassembles des exemples, tu acceptes une erreur moyenne, tu te donnes les moyens de la mesurer, tu décides si cette erreur est tolérable pour le cas d'usage. Beaucoup de frustration vient d'attendre la perfection là où il fallait un score utile avec un humain dans la boucle.

Quand Noé prédit un risque de retour, il ne remplace pas le service client. Il priorise. Quand un hôpital utilise un score (hors du périmètre de ce livre introductif, et avec des cadres stricts), l'enjeu n'est plus le même : les coûts d'erreur explosent, les biais deviennent critiques, la gouvernance monte. Adapte toujours la profondeur de ta démarche à l'impact. Un modèle jouet sur un CSV public n'exige pas la même revue qu'un score qui bloque un crédit.

Données : le personnage principal

Les algorithmes changent. Les principes de données restent : définition claire, représentativité, fraîcheur, droits, documentation, absence de fuite, inspection des cas bizarres. Passe plus de temps sur les données que sur le shopping d'algorithmes. C'est le conseil le moins glamour et le plus rentable du livre. Un arbre simple sur des features excellentes bat une usine à gaz sur un tableau sale.

Mise en production (aperçu)

Un notebook n'est pas un produit. En production, tu dois gérer des entrées manquantes nouvelles, des catégories inconnues, des délais, des journaux, des versions, des rollback, des alertes si la métrique chute. Tu n'as pas à tout construire aujourd'hui. Tu dois savoir que ça existe, pour ne pas crier victoire trop tôt après un score de validation. Prévois dès le jour 1 un chemin "humain si doute".

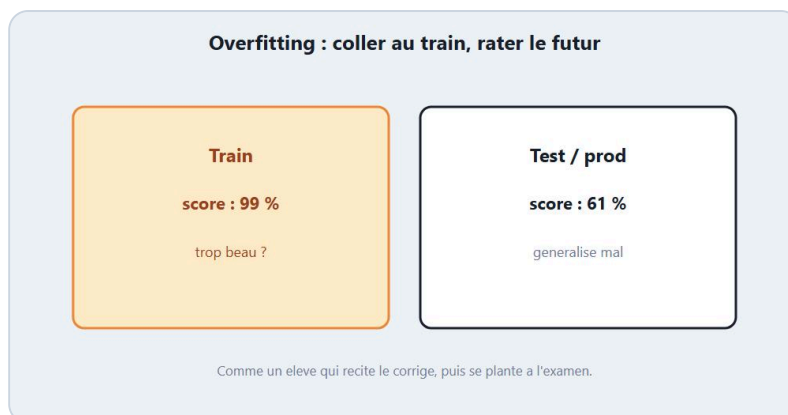
Culture et communication

Apprends a dire "non" a un modele inutile. Apprends a dire "pas encore" quand les labels manquent. Apprends a dire "voici les limites" quand tu presentes un score. Cette honnetete te rend plus credible que n'importe quel jargon. Chez DanielCraft, on forme des gens capables de tenir cette conversation avec un metier, un manager, ou un client.

Pour aller plus loin sans te perdre

Tu n'as pas a tout maitriser d'un coup. Reviens a ce chapitre quand tu butes sur un cas reel. Relis l'erreur classique. Refais le "A toi". Chez DanielCraft, on prefere trois relectures actives a une lecture passive de vingt pages. Si un paragraphe te semble encore flou, reformule-le a voix haute avec ton propre exemple. Des que ca passe a l'oral, c'est que c'est entre. Ensuite seulement, passe au chapitre suivant. Cette discipline lente cree une competence rapide sur la duree - le contraire du binge de tutos oublies le lendemain.

Chapitre 9 - Overfitting : trop coller a l'entrainement



Overfitting : coller au train, rater le futur.

L'**overfitting**, c'est quand le modele apprend par coeur les particularites du train (y compris le bruit) et generalise mal sur des cas nouveaux. Image : un etudiant qui recite le corrige des exercices deja vus et se plante a l'examen. Score train splendide. Score test decevant. En production, encore pire.

★ A RETENIR

Overfitting = coller au train, rater le futur. Un 100 % train est souvent un aveu, pas un tropee.

Sous-apprentissage aussi

A l'inverse, l'**underfitting**, c'est un modele trop simple qui rate meme les motifs reels. Score train mediocre, test mediocre. Tu veux la zone ou le modele capture le signal sans epouser le bruit.

Signes qui doivent t'alerter

Ecart **train/test** large. Arbre tres profond. Trop de **features** par rapport aux exemples. Performances magiques peu believable. Importance d'IDs ou de timestamps suspects. Forte sensibilite a une petite perturbation des donnees.

⚠ ATTENTION

Si le score train est excellent et le test mediocre, arrete d'ajouter de la complexite. Simplifie, regularise, ou revois les donnees.

Remedes pratiques

Plus de donnees utiles (pas seulement plus de lignes dupliques). **Regularisation** / profondeur limitee. Moins de features pourries. Ensembles d'arbres bien regles. Early stopping dans certains apprentissages. Surtout : protocole train/test honnete et inspection des erreurs. Parfois le vrai remede est de simplifier la question metier.

♦ ASTUCE

Compare toujours train et test cote a cote. L'ecart raconte plus qu'un seul chiffre isole.

Lien avec l'IA generative

Les LLM aussi peuvent "surapprendre" des styles ou des faits memorises, mais le vocabulaire et les outils different. Ici, reste sur l'image examen : ce qui compte, c'est la performance hors des exemples vus, dans des conditions proches du futur.



Exemple : Lea compare le score train et le score test.

Erreur classique

Ajouter de la complexite des qu'un score train n'est pas 100 %. Le 100 % train est souvent un aveu, pas un trophée.

A toi

Imagine un modele qui predit parfaitement ton historique. Donne 3 raisons pour lesquelles il pourrait echouer la semaine prochaine.

Regularisation en mots simples

Freiner la complexite : penaliser les poids trop grands, limiter la profondeur, exiger plus d'exemples par feuille d'arbre, arreter l'apprentissage tot. Ce n'est pas "rendre le modele bete". C'est l'empêcher d'apprendre le bruit.

Developpement : penser comme un artisan des modeles

Le machine learning n'est pas un distributeur de verite. C'est un artisanat de decisions sous incertitude. Tu choisis une question, tu rassembles des exemples, tu acceptes une erreur moyenne, tu te donnes les moyens de la mesurer, tu decides si cette erreur est tolerable pour le cas d'usage. Beaucoup de frustration vient d'attendre la perfection la ou il fallait un score utile avec un humain dans la boucle.

Quand Noe predit un risque de retour, il ne remplace pas le service client. Il priorise. Quand un hopital utilise un score (hors du perimetre de ce livre introductif, et avec des cadres stricts), l'enjeu n'est plus le meme : les couts d'erreur explosent, les biais deviennent critiques, la gouvernance monte. Adapte toujours la profondeur de ta demarche a l'impact. Un modele jouet sur un CSV public n'exige pas la meme revue qu'un score qui bloque un credit.

Donnees : le personnage principal

Les algorithmes changent. Les principes de donnees restent : definition claire, representativite, fraicheur, droits, documentation, absence de fuite, inspection des cas bizarres. Passe plus de temps sur les donnees que sur le shopping d'algorithmes. C'est le conseil le moins glamour et le plus rentable du livre. Un arbre simple sur des features excellentes bat une usine a gaz sur un tableau sale.

Mise en production (aperçu)

Un notebook n'est pas un produit. En production, tu dois gerer des entrees manquantes nouvelles, des categories inconnues, des delais, des journaux, des versions, des rollback, des alertes si la metrique chute. Tu n'as pas a tout construire aujourd'hui. Tu dois savoir que ca existe, pour ne pas crier victoire trop tot apres un score de validation. Prevour des le jour 1 un chemin "humain si doute".

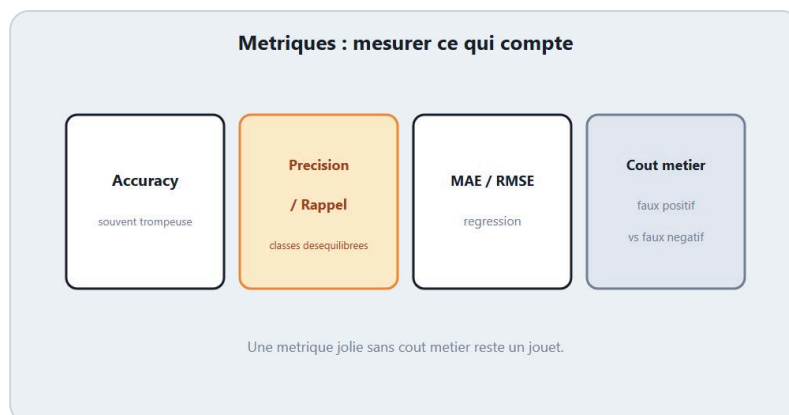
Culture et communication

Apprends a dire "non" a un modele inutile. Apprends a dire "pas encore" quand les labels manquent. Apprends a dire "voici les limites" quand tu presentes un score. Cette honnetete te rend plus credible que n'importe quel jargon. Chez DanielCraft, on forme des gens capables de tenir cette conversation avec un metier, un manager, ou un client.

Pour aller plus loin sans te perdre

Tu n'as pas a tout maitriser d'un coup. Reviens a ce chapitre quand tu butes sur un cas reel. Relis l'erreur classique. Refais le "A toi". Chez DanielCraft, on prefere trois relectures actives a une lecture passive de vingt pages. Si un paragraphe te semble encore flou, reformule-le a voix haute avec ton propre exemple. Des que ca passe a l'oral, c'est que c'est entre. Ensuite seulement, passe au chapitre suivant. Cette discipline lente cree une competence rapide sur la duree - le contraire du binge de tutos oublies le lendemain.

Chapitre 10 - Metriques : mesurer sans se raconter d'histoires



Metriques : accuracy, precision, rappel, cout metier.

Une **metrique** traduit une erreur en nombre. Mauvaise metrique, mauvaise decision. La "bonne" metrique depend du cout metier, pas de la mode GitHub.

★ A RETENIR

Choisis la metrique pour le cout metier, pas pour impressionner. Traduis toujours le score en decision.

Regression

MAE (erreur absolue moyenne) : lisible en unites de y. **RMSE** : penalise plus les grosses erreurs. **MAPE** : en pourcentage, attention aux y proches de zero. **R2** : part de variance expliquee, utile mais pas sacre. Regarde aussi des graphiques residus : les nombres seuls mentent parfois.

Classification

Accuracy : part de bons labels - trompeuse si classes desequilibrees. **Precision** : parmi les alertes, combien sont justes. **Rappel** (recall) : parmi les vrais positifs, combien tu as attrapes. **F1** : compromis. Matrice de confusion : le tableau qui montre ou tu te trompes. Courbes et **AUC** : utiles, a condition de comprendre le seuil ensuite.

⚠ ATTENTION

Sur classes desequilibrees, l'accuracy peut mentir. Pair precision/rappel (ou couts) avant de celebrer.

Cout metier > score unique

Si manquer une fraude coute 1000 et deranger un client sain coute 10, tu n'optimises pas comme si les erreurs etaient egales. Ecris une petite matrice de couts. Choisis seuil et modele en consequence. Noe prefere parfois plus de fausses alertes de retour que rater des retours massifs sur une gamme - ou l'inverse, selon sa logistique.

♦ ASTUCE

Une metrique principale + une metrique de garde-fou. La premiere decide, la seconde empeche les derives.

Baseline toujours

Compare a "toujours predire la moyenne" ou "toujours la classe majoritaire". Un modele complique qui bat a peine le **baseline** n'est pas un succes. Un modele simple qui le bat nettement, si.

Erreur classique

Chasser le 0.01 d'accuracy en ignorant que le label a change de definition. Ou presenter un score sans intervalle, sans taille d'echantillon, sans contexte de deploiement.

A toi

Choisis UNE metrique principale et UNE metrique de garde-fou pour ton projet. Justifie en trois phrases metier.

Raconter un score a un non-tech

Evite "notre F1 est 0,73". Prefere : "sur 100 vrais retours, on en attrape 80, et sur 100 alertes, 40 sont des fausses peurs ; voici ce que ca coute". Traduis. Si tu ne peux pas traduire, tu ne choisis peut-etre pas la bonne metrique.

Developpement : penser comme un artisan des modeles

Le machine learning n'est pas un distributeur de verite. C'est un artisanat de decisions sous incertitude. Tu choisis une question, tu rassembles des exemples, tu acceptes une erreur moyenne, tu te donnes les moyens de la mesurer, tu decides si cette erreur est tolerable pour le cas d'usage. Beaucoup de frustration vient d'attendre la perfection la ou il fallait un score utile avec un humain dans la boucle.

Quand Noe predit un risque de retour, il ne remplace pas le service client. Il priorise. Quand un hopital utilise un score (hors du perimetre de ce livre introductif, et avec des cadres stricts), l'enjeu n'est plus le meme : les couts d'erreur explosent, les biais deviennent critiques, la gouvernance monte. Adapte toujours la profondeur de ta demarche a l'impact. Un modele jouet sur un CSV public n'exige pas la meme revue qu'un score qui bloque un credit.

Donnees : le personnage principal

Les algorithmes changent. Les principes de donnees restent : definition claire, representativite, fraicheur, droits, documentation, absence de fuite, inspection des cas bizarres. Passe plus de temps sur les donnees que sur le shopping d'algorithmes. C'est le conseil le moins glamour et le plus rentable du livre. Un arbre simple sur des features excellentes bat une usine a gaz sur un tableau sale.

Mise en production (aperçu)

Un notebook n'est pas un produit. En production, tu dois gerer des entrees manquantes nouvelles, des categories inconnues, des delais, des journaux, des versions, des rollback, des alertes si la metrique chute. Tu n'as pas a tout construire aujourd'hui. Tu dois savoir que ca existe, pour ne pas crier victoire trop tot apres un score de validation. Prevour des le jour 1 un chemin "humain si doute".

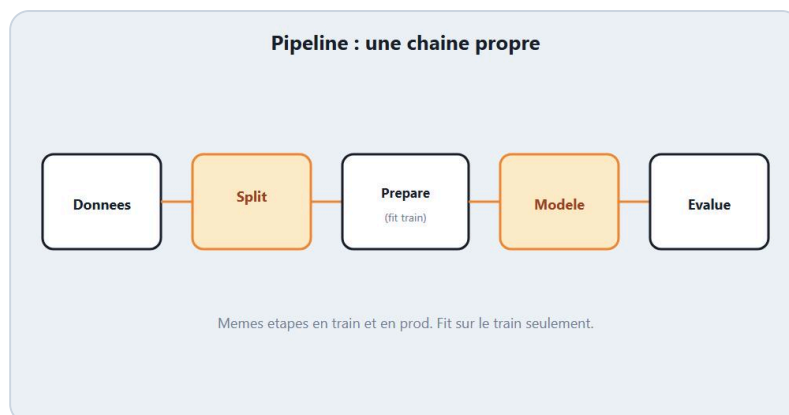
Culture et communication

Apprends a dire "non" a un modele inutile. Apprends a dire "pas encore" quand les labels manquent. Apprends a dire "voici les limites" quand tu presentes un score. Cette honnetete te rend plus credible que n'importe quel jargon. Chez DanielCraft, on forme des gens capables de tenir cette conversation avec un metier, un manager, ou un client.

Pour aller plus loin sans te perdre

Tu n'as pas a tout maitriser d'un coup. Reviens a ce chapitre quand tu butes sur un cas reel. Relis l'erreur classique. Refais le "A toi". Chez DanielCraft, on prefere trois relectures actives a une lecture passive de vingt pages. Si un paragraphe te semble encore flou, reformule-le a voix haute avec ton propre exemple. Des que ca passe a l'oral, c'est que c'est entre. Ensuite seulement, passe au chapitre suivant. Cette discipline lente cree une competence rapide sur la duree - le contraire du binge de tutos oublies le lendemain.

Chapitre 11 - Pipeline : enchaîner proprement les étapes



Pipeline : donnees, prepare, modele, evaluation.

Un **pipeline**, c'est une chaîne : préparation des **features**, modèle, prédiction. L'intérêt n'est pas cosmétique. C'est d'éviter la triche et le chaos : les mêmes transformations au train et au test, dans le bon ordre, sans fuite, reproductibles.

★ A RETENIR

Pipeline = même chaîne train et prod, fit sur le train seulement. Moins de fuite, plus de clarté.

Étapes typiques

1) Lecture des données. 2) Split. 3) Traitement des manquants (ajuste sur train). 4) Encodage des catégories. 5) Scaling si besoin. 6) Modèle. 7) Évaluation. 8) Export du modèle + des transformateurs. En production, tu rejoues la même chaîne sur une nouvelle ligne.

Pourquoi scikit-learn aime ça

Dans l'esprit **scikit-learn** (chapitre suivant), on branche des étapes qui exposent **fit** / **transform** / **predict**. Tu fais fit sur le train, transform sur le test, predict. Si tu ajoutes une étape, tu l'insères dans la chaîne, tu ne recopies pas six scripts divergents. Moins d'erreurs humaines. Plus de clarté.

♦ ASTUCE

Dessine le pipeline en boîtes avant de coder. Marque clairement où se fait le fit et où se fait le transform.

Reproductibilité

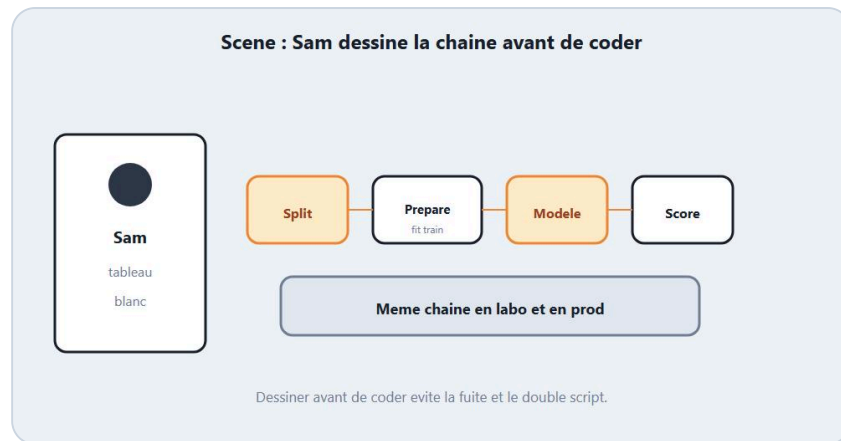
Fixe les graines aléatoires quand c'est pertinent. Versionne le code et note la version des données. Documente les définitions de features. Un modèle non reproductible est un souvenir, pas un actif.

Monitoring après déploiement

Les données dérivent : nouveaux produits, nouvelle saison, nouveau comportement. Surveille les distributions de features et les **metriques** dans le temps. Prévois un reentrainement. Un pipeline sans monitoring est une fusée sans tableau de bord.

⚠ ATTENTION

Un notebook de recherche different du script de prod, c'est le piege classique. Unifie tot, sinon le score labo ne veut plus rien dire.



Exemple : Sam dessine la chaine avant de coder.

Erreur classique

Preprocessing "a la main" different entre le notebook de recherche et le script de prod. Le score labo ne veut plus rien dire. Unifie tot.

A toi

Dessine ton pipeline en 6 boites. Marque ou se fait le fit, ou se fait le transform, ou se calcule la metrique.

Tests du pipeline

Meme sans framework lourd : une ligne typique traverse la chaine ; une ligne avec manquants ; une categorie inconnue ; une date future. Si ca casse en silence, tu le sauras trop tard. Automatise ces cas des que tu peux.

Developpement : penser comme un artisan des modeles

Le machine learning n'est pas un distributeur de verite. C'est un artisanat de decisions sous incertitude. Tu choisis une question, tu rassembles des exemples, tu acceptes une erreur moyenne, tu te donnes les moyens de la mesurer, tu decides si cette erreur est tolerable pour le cas d'usage. Beaucoup de frustration vient d'attendre la perfection la ou il fallait un score utile avec un humain dans la boucle.

Quand Noe predit un risque de retour, il ne remplace pas le service client. Il priorise. Quand un hopital utilise un score (hors du perimetre de ce livre introductif, et avec des cadres stricts), l'enjeu n'est plus le meme : les couts d'erreur explosent, les biais deviennent critiques, la gouvernance monte. Adapte toujours la profondeur de ta demarche a l'impact. Un modele jouet sur un CSV public n'exige pas la meme revue qu'un score qui bloque un credit.

Donnees : le personnage principal

Les algorithmes changent. Les principes de donnees restent : definition claire, representativite, fraicheur, droits, documentation, absence de fuite, inspection des cas bizarres. Passe plus de temps sur les donnees que sur le shopping d'algorithmes. C'est le conseil le moins glamour et le plus rentable du livre. Un arbre simple sur des features excellentes bat une usine a gaz sur un tableau sale.

Mise en production (aperçu)

Un notebook n'est pas un produit. En production, tu dois gerer des entrees manquantes nouvelles, des categories inconnues, des delais, des journaux, des versions, des rollback, des alertes si la metrique chute. Tu n'as pas a tout construire aujourd'hui. Tu dois savoir que ca existe, pour ne pas crier victoire trop tot apres un score de validation. Prevour des le jour 1 un chemin "humain si doute".

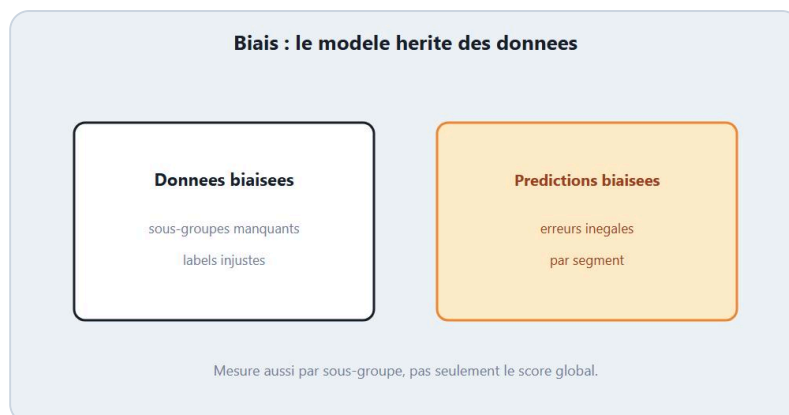
Culture et communication

Apprends a dire "non" a un modele inutile. Apprends a dire "pas encore" quand les labels manquent. Apprends a dire "voici les limites" quand tu presentes un score. Cette honnetete te rend plus credible que n'importe quel jargon. Chez DanielCraft, on forme des gens capables de tenir cette conversation avec un metier, un manager, ou un client.

Pour aller plus loin sans te perdre

Tu n'as pas a tout maitriser d'un coup. Reviens a ce chapitre quand tu butes sur un cas reel. Relis l'erreur classique. Refais le "A toi". Chez DanielCraft, on prefere trois relectures actives a une lecture passive de vingt pages. Si un paragraphe te semble encore flou, reformule-le a voix haute avec ton propre exemple. Des que ca passe a l'oral, c'est que c'est entre. Ensuite seulement, passe au chapitre suivant. Cette discipline lente cree une competence rapide sur la duree - le contraire du binge de tutos oublies le lendemain.

Chapitre 12 - Biais dans les données : le modèle hérité



Données biaisées = prédictions biaisées.

Un modèle **supervisé** apprend ce que tu lui montres. Si tes exemples sous-représentent une population, si tes **labels** portent un préjugé historique, si tes mesures sont biaisées, le modèle industrialise le problème - souvent plus vite et à plus grande échelle. Ce n'est pas "la faute de l'algorithme" comme créature malveillante. C'est un miroir statistique + une décision de déploiement.

★ A RETENIR

Le modèle hérité des **biais** des données. Un beau score global ne lave pas une injustice locale.

Types de biais utiles à connaître

Biais de sélection : tu n' observes que ceux qui sont entrés dans ton système. Biais de label : les annotateurs (ou l'historique) favorisent une classe. Biais de mesure : un capteur ou un process sous-estime un groupe. Biais de feedback : le modèle influence le monde, qui renvoie des données qui confirment le modèle. Chez DanielCraft, on demande toujours : qui manque dans le jeu ? Qui a labellisé ? Qui paie l'erreur ?

⚠ ATTENTION

Retirer une variable sensible ne suffit pas toujours : d'autres **features** peuvent la reconstruire. Vérifie les performances par sous-groupe.

Exemple Noe

Si Noe n'a historiquement que des clients urbains aisés et déploie un score de retour partout, il peut mal traiter d'autres segments. Si les retours textiles sont sur-labels "abusifs" à cause d'un a priori d'équipe, le classifieur apprend l'a priori. Corriger, ce n'est pas seulement "ajouter de l'éthique en slide" : c'est revoir les labels, l'échantillonnage, **metriques** par segment, et parfois refuser d'automatiser.

Que faire concrètement

Mesurer les performances par sous-groupes pertinents. Inspecter les erreurs. Interroger la représentativité. Documenter les limites. Garder un humain sur les décisions à fort impact. Éviter les proxies dangereux (utiliser un code postal pour estimer "fiabilité" de façon opaque). Préférer la transparence à la magie.

♦ ASTUCE

Avant deploiement a impact : metriques par groupe, recours humain, doc des limites, qui signe le go.
Quatre questions, pas une slide ethique.

Lien legal et moral

Selon le domaine (credit, emploi, sante...), des cadres existent. Ce livre ne remplace pas un conseil juridique. Il pose le reflexe : un beau score global ne lave pas une injustice locale. Si tu deploies, assume.

Erreur classique

Croire que retirer une variable sensible (genre, origine) suffit toujours : d'autres features la reconstruisent. Ou croire que "c'est des maths donc neutre". Les maths operent sur un monde deja charge.

A toi

Pour ton projet, nomme un sous-groupe qui pourrait etre mal servi. Quelle metrique regarderas-tu separement ?

Processus de revue

Avant deploiement a impact : qui est affecte ? Quelles metriques par groupe ? Quel recours humain ? Quelle doc des limites ? Qui signe le go ? Ce n'est pas de la paperasse gratuite. C'est ce qui distingue un prototype jouet d'un systeme responsable.

Developpement : penser comme un artisan des modeles

Le machine learning n'est pas un distributeur de verite. C'est un artisanat de decisions sous incertitude. Tu choisis une question, tu rassembles des exemples, tu acceptes une erreur moyenne, tu te donnes les moyens de la mesurer, tu decides si cette erreur est tolerable pour le cas d'usage. Beaucoup de frustration vient d'attendre la perfection la ou il fallait un score utile avec un humain dans la boucle.

Quand Noe predit un risque de retour, il ne remplace pas le service client. Il priorise. Quand un hopital utilise un score (hors du perimetre de ce livre introductif, et avec des cadres stricts), l'enjeu n'est plus le meme : les couts d'erreur explosent, les biais deviennent critiques, la gouvernance monte. Adapte toujours la profondeur de ta demarche a l'impact. Un modele jouet sur un CSV public n'exige pas la meme revue qu'un score qui bloque un credit.

Donnees : le personnage principal

Les algorithmes changent. Les principes de donnees restent : definition claire, representativite, fraicheur, droits, documentation, absence de fuite, inspection des cas bizarres. Passe plus de temps sur les donnees que sur le shopping d'algorithmes. C'est le conseil le moins glamour et le plus rentable du livre. Un arbre simple sur des features excellentes bat une usine a gaz sur un tableau sale.

Mise en production (aperçu)

Un notebook n'est pas un produit. En production, tu dois gerer des entrees manquantes nouvelles, des categories inconnues, des delais, des journaux, des versions, des rollback, des alertes si la metrique chute. Tu n'as pas a tout construire aujourd'hui. Tu dois savoir que ca existe, pour ne pas crier victoire trop tot apres un score de validation. Prevour des le jour 1 un chemin "humain si doute".

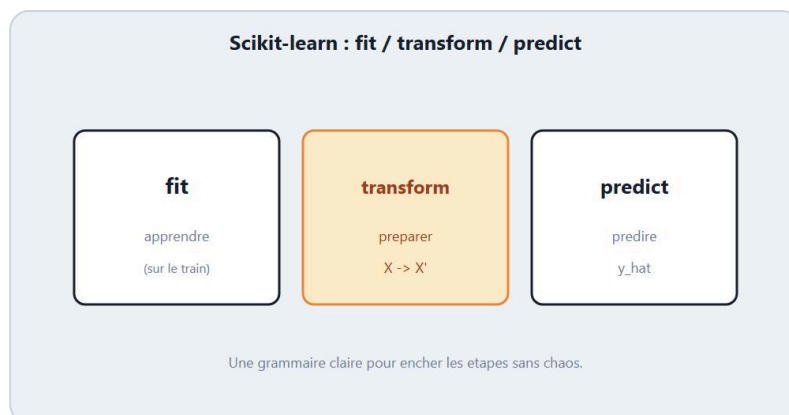
Culture et communication

Apprends a dire "non" a un modele inutile. Apprends a dire "pas encore" quand les labels manquent. Apprends a dire "voici les limites" quand tu presentes un score. Cette honnetete te rend plus credible que n'importe quel jargon. Chez DanielCraft, on forme des gens capables de tenir cette conversation avec un metier, un manager, ou un client.

Pour aller plus loin sans te perdre

Tu n'as pas a tout maitriser d'un coup. Reviens a ce chapitre quand tu butes sur un cas reel. Relis l'erreur classique. Refais le "A toi". Chez DanielCraft, on prefere trois relectures actives a une lecture passive de vingt pages. Si un paragraphe te semble encore flou, reformule-le a voix haute avec ton propre exemple. Des que ca passe a l'oral, c'est que c'est entre. Ensuite seulement, passe au chapitre suivant. Cette discipline lente cree une competence rapide sur la duree - le contraire du binge de tutos oublies le lendemain.

Chapitre 13 - Scikit-learn : l'idée sans jargon opaque



Scikit-learn : fit, transform, predict.

Scikit-learn est une bibliothèque Python très utilisée pour le **machine learning** "classique". Tu n'es pas obligé de coder aujourd'hui pour comprendre son esprit. L'idée : une grammaire commune pour splitter, transformer, entraîner, prédire, évaluer. Les noms d'algorithmes changent ; le verbe reste : **fit**, **transform**, **predict**, score.

★ A RETENIR

Scikit-learn = grammaire fit / transform / predict. La discipline compte plus que le nom de l'algo.

La grammaire mentale

Tu as un tableau X (lignes = exemples, colonnes = **features**) et souvent un vecteur y . Tu crées un objet modèle. Tu appelles `fit(X_train, y_train)` : il apprend. Tu appelles `predict(X_test)` : il propose. Tu calcules une **metrique** entre y_{test} et les prédictions. Pour les transformateurs (imputation, scaling, encodage), `fit` apprend les paramètres sur le train, `transform` applique.

Pipeline dans scikit-learn

Tu enchaînes transformateurs + modèle dans un **Pipeline**. Un seul `fit` sur le train. Un seul `predict` sur le nouveau. Moins de fuite. Plus propre. C'est exactement le chapitre pipeline mis en code. Même si tu utilises un autre outil plus tard, cette discipline reste.

♦ ASTUCE

Sans écrire de code, décris mentalement : colonnes de X , colonne y , premier modèle simple, métrique. Si tu codes, commence par battre une baseline.

Ce que scikit-learn fait bien

Modèles solides pour tableaux (régression, classification, clustering). Outils de validation croisée, recherche de réglages, métriques. Excellent pour apprendre. Limites : ce n'est pas le cœur du deep learning à grande échelle (autres bibliothèques), ni le traitement du langage type LLM. Pour Noe sur un CSV de commandes, c'est souvent le bon terrain.

⚠ ATTENTION

Ne scale pas sur tout le dataset avant le split. Fit des transformateurs sur le train uniquement - meme piege, meme regle.

Mini parcours d'apprentissage code (optionnel)

1) Charger un CSV. 2) Separer train/test. 3) Pipeline simple (encodage + modele lineaire ou arbre). 4) Score. 5) Matrice de confusion ou erreurs. 6) Inspecter. Tu peux suivre n'importe quel tutoriel officiel en gardant les idees de ce livre comme boussole anti-pieges.

Erreur classique

Copier un notebook qui scale sur tout le dataset avant split. Ou choisir un modele complexe parce que le nom impressionne. Ou oublier de fixer le traitement des categories inconnues en production.

A toi

Sans ecrire de code, decris l'appel mental : quelles colonnes dans X, quelle colonne y, quel modele simple en premier, quelle metrique. Si tu codes deja, implemente la baseline cette semaine.

Ecosysteme voisin

Pandas pour les tableaux, matplotlib/seaborn pour voir, jupyter pour explorer, puis un script propre pour produire. Scikit-learn se place au milieu : modeles et metriques. Plus tard, d'autres outils viendront ; la discipline fit/transform restera.

Developpement : penser comme un artisan des modeles

Le machine learning n'est pas un distributeur de verite. C'est un artisanat de decisions sous incertitude. Tu choisis une question, tu rassembles des exemples, tu acceptes une erreur moyenne, tu te donnes les moyens de la mesurer, tu decides si cette erreur est tolerable pour le cas d'usage. Beaucoup de frustration vient d'attendre la perfection la ou il fallait un score utile avec un humain dans la boucle.

Quand Noe predit un risque de retour, il ne remplace pas le service client. Il priorise. Quand un hopital utilise un score (hors du perimetre de ce livre introductif, et avec des cadres stricts), l'enjeu n'est plus le meme : les couts d'erreur explosent, les biais deviennent critiques, la gouvernance monte. Adapte toujours la profondeur de ta demarche a l'impact. Un modele jouet sur un CSV public n'exige pas la meme revue qu'un score qui bloque un credit.

Donnees : le personnage principal

Les algorithmes changent. Les principes de donnees restent : definition claire, representativite, fraicheur, droits, documentation, absence de fuite, inspection des cas bizarres. Passe plus de temps sur les donnees que sur le shopping d'algorithmes. C'est le conseil le moins glamour et le plus rentable du livre. Un arbre simple sur des features excellentes bat une usine a gaz sur un tableau sale.

Mise en production (aperçu)

Un notebook n'est pas un produit. En production, tu dois gérer des entrées manquantes nouvelles, des catégories inconnues, des délais, des journaux, des versions, des rollback, des alertes si la métrique chute. Tu n'as pas à tout construire aujourd'hui. Tu dois savoir que ça existe, pour ne pas crier victoire trop tôt après un score de validation. Prévois dès le jour 1 un chemin "humain si doute".

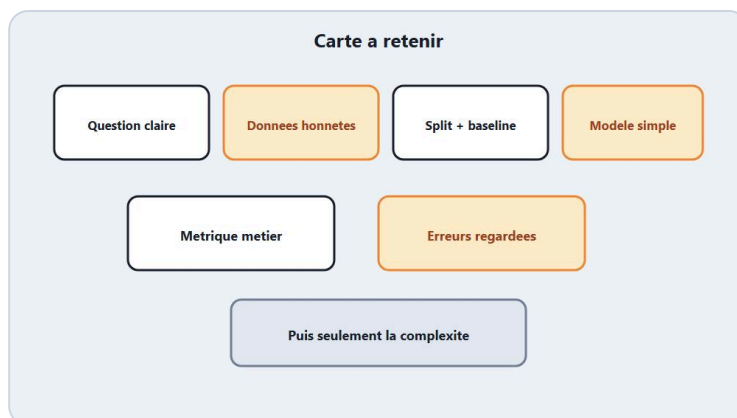
Culture et communication

Apprends à dire "non" à un modèle inutile. Apprends à dire "pas encore" quand les labels manquent. Apprends à dire "voici les limites" quand tu présentes un score. Cette honnêteté te rend plus crédible que n'importe quel jargon. Chez DanielCraft, on forme des gens capables de tenir cette conversation avec un métier, un manager, ou un client.

Pour aller plus loin sans te perdre

Tu n'as pas à tout maîtriser d'un coup. Reviens à ce chapitre quand tu butes sur un cas réel. Relis l'erreur classique. Refais le "A toi". Chez DanielCraft, on préfère trois relectures actives à une lecture passive de vingt pages. Si un paragraphe te semble encore flou, reformule-le à voix haute avec ton propre exemple. Dès que ça passe à l'oral, c'est que c'est entre. Ensuite seulement, passe au chapitre suivant. Cette discipline lente crée une compétence rapide sur la durée - le contraire du binge de tutos oubliés le lendemain.

Chapitre 14 - A retenir



Boucle : question, donnees, split, baseline, metrique.

Machine learning = apprendre a partir d'exemples. **Supervise** = avec labels. Non supervise = structures sans labels. **Regression** = nombre. Classification = classe. Arbres = questions en cascade. **Features** = ce que le modele voit, disponibles au bon moment. **Train/test** = ne pas se mentir. **Overfitting** = coller au train, rater le futur. Metriques = alignees au cout metier. **Pipeline** = chaines propres sans fuite. Biais = le modele herite des donnees. Scikit-learn = grammaire fit / transform / predict.

Ce chapitre n'invente rien de nouveau. Il compacte le parcours pour que tu puisses y revenir comme a une fiche murale. Si une ligne te parait encore floue, retourne au chapitre dedie plutot que de forcer. Chez DanielCraft, on prefere une boucle courte bien digeree a une encyclopedie demi-comprise.

★ A RETENIR

Question claire, donnees honnetes, split, baseline, modele simple, metrique metier, erreurs regardees - puis seulement la complexite.

Boucle DanielCraft

Question claire -> donnees honnetes -> split -> baseline -> modele simple -> metrique metier -> inspection des erreurs -> features -> seulement ensuite complexite -> surveillance apres deploiement.

♦ ASTUCE

Recopie la boucle en TES mots sur une page. Ajoute ton exemple (Noe ou le tien) a chaque etape. Une page bat dix slides.

Ce que tu peux oublier

La liste de tous les algorithmes a la mode. Garde les gestes. Change d'outil si besoin.

A toi

Recopie la boucle en TES mots sur une page. Ajoute ton exemple Noe (ou le tien) a chaque etape.

Checklist une minute

Question ? Label ? Instant T ? Split ? Baseline ? Metrique ? Erreurs regardees ? Biais ? Pipeline ? Monitoring ? Si tu coches, tu es deja au-dessus de beaucoup de demos LinkedIn.

♦ ASTUCE

Colle cette checklist pres de ton ecran. Avant chaque "victoire" de score, parcours-la en trente secondes.

Developpement : penser comme un artisan des modeles

Le machine learning n'est pas un distributeur de verite. C'est un artisanat de decisions sous incertitude. Tu choisis une question, tu rassembles des exemples, tu acceptes une erreur moyenne, tu te donnes les moyens de la mesurer, tu decides si cette erreur est tolerable pour le cas d'usage. Beaucoup de frustration vient d'attendre la perfection la ou il fallait un score utile avec un humain dans la boucle.

Quand Noe predit un risque de retour, il ne remplace pas le service client. Il priorise. Quand un hopital utilise un score (hors du perimetre de ce livre introductif, et avec des cadres stricts), l'enjeu n'est plus le meme : les couts d'erreur explosent, les biais deviennent critiques, la gouvernance monte. Adapte toujours la profondeur de ta demarche a l'impact. Un modele jouet sur un CSV public n'exige pas la meme revue qu'un score qui bloque un credit.

Donnees : le personnage principal

Les algorithmes changent. Les principes de donnees restent : definition claire, representativite, fraicheur, droits, documentation, absence de fuite, inspection des cas bizarres. Passe plus de temps sur les donnees que sur le shopping d'algorithmes. C'est le conseil le moins glamour et le plus rentable du livre. Un arbre simple sur des features excellentes bat une usine a gaz sur un tableau sale.

Mise en production (aperçu)

Un notebook n'est pas un produit. En production, tu dois gerer des entrees manquantes nouvelles, des categories inconnues, des delais, des journaux, des versions, des rollback, des alertes si la metrique chute. Tu n'as pas a tout construire aujourd'hui. Tu dois savoir que ca existe, pour ne pas crier victoire trop tot apres un score de validation. Prevour des le jour 1 un chemin "humain si doute".

Culture et communication

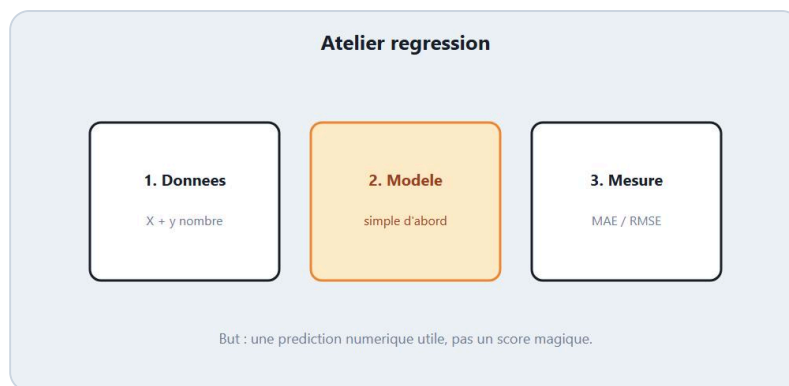
Apprends a dire "non" a un modele inutile. Apprends a dire "pas encore" quand les labels manquent. Apprends a dire "voici les limites" quand tu presentes un score. Cette honnetete te rend plus credible que n'importe quel jargon. Chez DanielCraft, on forme des gens capables de tenir cette conversation avec un metier, un manager, ou un client.

Pour aller plus loin sans te perdre

Tu n'as pas a tout maitriser d'un coup. Reviens a ce chapitre quand tu butes sur un cas reel. Relis l'erreur classique. Refais le "A toi". Chez DanielCraft, on prefere trois relectures actives a une lecture passive de vingt pages. Si un paragraphe te semble encore flou, reformule-le a voix haute avec ton propre exemple. Des que ca passe a l'oral, c'est que c'est entre. Ensuite seulement, passe au chapitre suivant. Cette

discipline lente cree une competence rapide sur la duree - le contraire du binge de tutos oublies le lendemain.

Chapitre 15 - Atelier : une regression de bout en bout (sur papier ou code)



Atelier regression : donnees, modele, mesure.

Objectif : mener une mini **regression** sans te perdre. Duree : 45 minutes. Tu n'as pas besoin d'un dataset parfait. Tu as besoin de vivre le geste : cibler un nombre, lister des **features** disponibles a l'instant T, splitter, battre une **baseline**, lire une **metrique**.

Le but n'est pas d'impressionner. C'est de sortir avec un "contrat regression" que tu pourrais montrer a un collegue. Si tu codes, tant mieux. Si tu restes sur papier avec quinze lignes inventees, le muscle s'active pareil.

♦ ASTUCE

Si tu n'as pas de donnees, invente 15 lignes coherentes a la main. Le geste compte plus que le volume.

Etapes

1) Choisis une cible numerique (ventes, duree, montant). 2) Liste 5 features disponibles a l'instant T. 3) Decris un split temporel ou aleatoire. 4) Definir la baseline (predire la moyenne train). 5) Imagine une regression lineaire a une feature principale : quelle pente sensee ? 6) Choisis MAE ou RMSE comme metrique. 7) Ecris 3 causes possibles d'**overfitting** sur ce sujet. 8) Si tu codes : implemente avec un **Pipeline** minimal et compare au baseline.

Livrable

Une page "contrat regression" : question, features, split, metrique, baseline, critere de succes ("battre baseline de X sur test").

Conseil

Si tu n'as pas de donnees, invente 15 lignes coherentes a la main pour jouer le geste. Le geste compte.

⚠ ATTENTION

N'utilise aucune info connue seulement apres la prediction. Sinon ton atelier s'entraine a la fuite.

Prolongement

Change une feature, recalcule mentalement l'effet sur la prediction. Ecris une phrase d'interpretation prudente ("associe a", pas "cause"). Ajoute-la au livrable.

Quand tu as fini, relis ton contrat a voix haute. Si tu ne peux pas expliquer le split et la metrique en trente secondes, le livrable n'est pas encore clair. Chez DanielCraft, on valide l'atelier quand le contrat tient sans jargon.

Developpement : penser comme un artisan des modeles

Le machine learning n'est pas un distributeur de verite. C'est un artisanat de decisions sous incertitude. Tu choisis une question, tu rassembles des exemples, tu acceptes une erreur moyenne, tu te donnes les moyens de la mesurer, tu decides si cette erreur est tolerable pour le cas d'usage. Beaucoup de frustration vient d'attendre la perfection la ou il fallait un score utile avec un humain dans la boucle.

Quand Noe predit un risque de retour, il ne remplace pas le service client. Il priorise. Quand un hopital utilise un score (hors du perimetre de ce livre introductif, et avec des cadres stricts), l'enjeu n'est plus le meme : les couts d'erreur explosent, les biais deviennent critiques, la gouvernance monte. Adapte toujours la profondeur de ta demarche a l'impact. Un modele jouet sur un CSV public n'exige pas la meme revue qu'un score qui bloque un credit.

Donnees : le personnage principal

Les algorithmes changent. Les principes de donnees restent : definition claire, representativite, fraicheur, droits, documentation, absence de fuite, inspection des cas bizarres. Passe plus de temps sur les donnees que sur le shopping d'algorithmes. C'est le conseil le moins glamour et le plus rentable du livre. Un arbre simple sur des features excellentes bat une usine a gaz sur un tableau sale.

Mise en production (aperçu)

Un notebook n'est pas un produit. En production, tu dois gerer des entrees manquantes nouvelles, des categories inconnues, des delais, des journaux, des versions, des rollback, des alertes si la metrique chute. Tu n'as pas a tout construire aujourd'hui. Tu dois savoir que ca existe, pour ne pas crier victoire trop tot apres un score de validation. Prevour des le jour 1 un chemin "humain si doute".

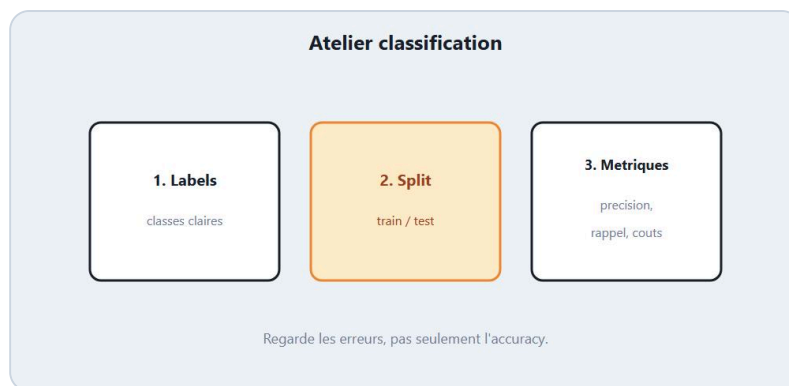
Culture et communication

Apprends a dire "non" a un modele inutile. Apprends a dire "pas encore" quand les labels manquent. Apprends a dire "voici les limites" quand tu presentes un score. Cette honnetete te rend plus credible que n'importe quel jargon. Chez DanielCraft, on forme des gens capables de tenir cette conversation avec un metier, un manager, ou un client.

Pour aller plus loin sans te perdre

Tu n'as pas a tout maitriser d'un coup. Reviens a ce chapitre quand tu butes sur un cas reel. Relis l'erreur classique. Refais le "A toi". Chez DanielCraft, on prefere trois relectures actives a une lecture passive de vingt pages. Si un paragraphe te semble encore flou, reformule-le a voix haute avec ton propre exemple. Des que ca passe a l'oral, c'est que c'est entre. Ensuite seulement, passe au chapitre suivant. Cette discipline lente cree une competence rapide sur la duree - le contraire du binge de tutos oublies le lendemain.

Chapitre 16 - Atelier : classification et cout des erreurs



Atelier classification : labels, split, metriques.

Objectif : concevoir un classifieur en pensant **faux positifs** / **faux negatifs**. Duree : 45 minutes. Ici, le score n'est pas le heros. Le heros, c'est le cout : qui souffre si tu te trompes d'un cote, qui souffre de l'autre.

Prends un cas binaire de ton monde (retour oui/non, fraude, ticket urgent...). Ecris les couts avant de toucher a un modele. Ensuite seulement, choisis un **seuil**, une **metrique**, et des **features** sans fuite. Tu sortiras avec une fiche de decision, pas avec un chiffre orphelin.

★ A RETENIR

Le seuil suit les couts. L'accuracy globale ne decide rien seule sur une classe rare.

Etapas

1) Choisis une cible binaire. 2) Ecris le cout d'un faux positif et d'un faux negatif. 3) Propose un seuil de decision (haut/bas) aligne a ces couts. 4) Dessine une **matrice de confusion** vide et imagine des effectifs. 5) Choisis precision ou rappel comme metrique principale + une de garde-fou. 6) Liste 5 features sans fuite temporelle. 7) Decris une **baseline** (classe majoritaire). 8) Option code : arbre peu profond + matrice de confusion.

Livrable

Fiche "decision classification" avec couts, seuil, metriques, features, plan d'inspection des erreurs (regarder 20 cas).

Piege a eviter

Optimiser l'**accuracy** globale alors que la classe rare est la seule qui compte.

⚠ ATTENTION

Si ta classe positive fait 5 %, un modele "toujours negatif" a 95 % d'accuracy et zero utilite. Regarde la detection de la classe rare.

Prolongement

Fais varier le seuil (bas / milieu / haut) et raconte pour chaque seuil qui gagne et qui perd. Choisis explicitement. Documente.

Quand tu as choisi, explique ton choix a voix haute comme si Noe te demandait pourquoi. Si tu bloques sur "parce que le score est mieux", recommence : le score doit servir une decision, pas l'inverse.

Developpement : penser comme un artisan des modeles

Le machine learning n'est pas un distributeur de verite. C'est un artisanat de decisions sous incertitude. Tu choisis une question, tu rassembles des exemples, tu acceptes une erreur moyenne, tu te donnes les moyens de la mesurer, tu decides si cette erreur est tolerable pour le cas d'usage. Beaucoup de frustration vient d'attendre la perfection la ou il fallait un score utile avec un humain dans la boucle.

Quand Noe predit un risque de retour, il ne remplace pas le service client. Il priorise. Quand un hopital utilise un score (hors du perimetre de ce livre introductif, et avec des cadres stricts), l'enjeu n'est plus le meme : les couts d'erreur explosent, les biais deviennent critiques, la gouvernance monte. Adapte toujours la profondeur de ta demarche a l'impact. Un modele jouet sur un CSV public n'exige pas la meme revue qu'un score qui bloque un credit.

Donnees : le personnage principal

Les algorithmes changent. Les principes de donnees restent : definition claire, representativite, fraicheur, droits, documentation, absence de fuite, inspection des cas bizarres. Passe plus de temps sur les donnees que sur le shopping d'algorithmes. C'est le conseil le moins glamour et le plus rentable du livre. Un arbre simple sur des features excellentes bat une usine a gaz sur un tableau sale.

Mise en production (aperçu)

Un notebook n'est pas un produit. En production, tu dois gerer des entrees manquantes nouvelles, des categories inconnues, des delais, des journaux, des versions, des rollback, des alertes si la metrique chute. Tu n'as pas a tout construire aujourd'hui. Tu dois savoir que ca existe, pour ne pas crier victoire trop tot apres un score de validation. Prevour des le jour 1 un chemin "humain si doute".

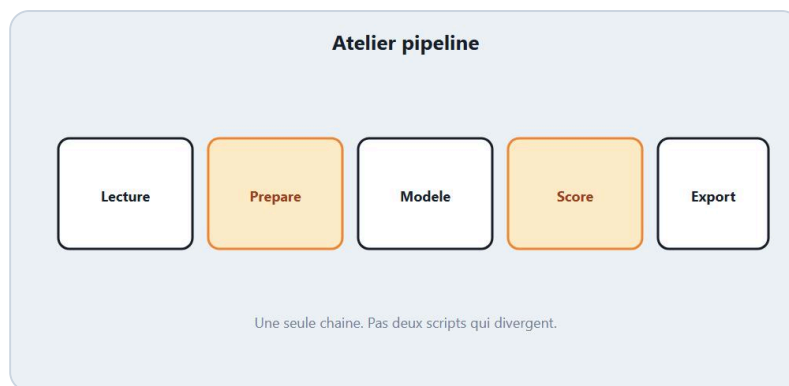
Culture et communication

Apprends a dire "non" a un modele inutile. Apprends a dire "pas encore" quand les labels manquent. Apprends a dire "voici les limites" quand tu presentes un score. Cette honnetete te rend plus credible que n'importe quel jargon. Chez DanielCraft, on forme des gens capables de tenir cette conversation avec un metier, un manager, ou un client.

Pour aller plus loin sans te perdre

Tu n'as pas a tout maitriser d'un coup. Reviens a ce chapitre quand tu butes sur un cas reel. Relis l'erreur classique. Refais le "A toi". Chez DanielCraft, on prefere trois relectures actives a une lecture passive de vingt pages. Si un paragraphe te semble encore flou, reformule-le a voix haute avec ton propre exemple. Des que ca passe a l'oral, c'est que c'est entre. Ensuite seulement, passe au chapitre suivant. Cette discipline lente cree une competence rapide sur la duree - le contraire du binge de tutos oublies le lendemain.

Chapitre 17 - Atelier : dessiner et valider un pipeline



Atelier pipeline : une seule chaine propre.

Objectif : produire un schema de **pipeline** anti-fuite. Duree : 30 minutes. Pas besoin de code pour cet atelier. Besoin d'un crayon et de l'honnete : chaque boite doit dire si elle voit le **train**, le **test**, ou les deux - et a quel moment.

Un pipeline clair, c'est ce qui separe un notebook qui "marche une fois" d'un systeme que tu oses rejouer en production. Tu vas marquer en rouge les tentations de fuite, lister les artefacts a sauver, et prevoir trois signaux de **monitoring**. A la fin, tu dois pouvoir expliquer la chaine a un collegue non tech.

♦ ASTUCE

Si une etape n'est pas assez claire pour l'expliquer a un non-tech, elle n'est pas prete. Simplifie la boite avant d'ajouter une fleche.

Etapes

1) Decoupe : raw -> split -> prep (**fit** train) -> modele -> **predict** -> **metrique**. 2) Marque en rouge toute etape qui utiliserait le test trop tot. 3) Ajoute une branche "nouvelles donnees en prod". 4) Liste les artefacts a sauvegarder (modele, encodeurs, version donnees). 5) Ajoute 3 signaux de monitoring (derive d'une feature, chute metrique, hausse des manquants). 6) Option : ecris un pseudo-code Pipeline style **scikit-learn**.

Livrable

Schema + checklist anti-fuite a 8 cases cochees.

Conseil DanielCraft

Si une etape n'est pas claire assez pour l'expliquer a un collegue non tech, elle n'est pas prete.

⚠ ATTENTION

Toute normalisation ou encodage "sur tout le CSV" avant le split est une fuite. Fit sur le train, transform ailleurs.

Prolongement

Ecris la liste des fichiers artefacts (modele.pkl, encodeur, config.yaml mental). Qui a le droit de les ecraser ? Ou vivent-ils ?

Une fois la liste posee, imagine la premiere alerte monitoring : que fais-tu dans l'heure ? Rollback, reentrainement, ou humain dans la boucle ? Note la reponse sur le schema. Un pipeline sans plan de secours reste une demo.

Developpement : penser comme un artisan des modeles

Le machine learning n'est pas un distributeur de verite. C'est un artisanat de decisions sous incertitude. Tu choisis une question, tu rassembles des exemples, tu acceptes une erreur moyenne, tu te donnes les moyens de la mesurer, tu decides si cette erreur est tolerable pour le cas d'usage. Beaucoup de frustration vient d'attendre la perfection la ou il fallait un score utile avec un humain dans la boucle.

Quand Noe predit un risque de retour, il ne remplace pas le service client. Il priorise. Quand un hopital utilise un score (hors du perimetre de ce livre introductif, et avec des cadres stricts), l'enjeu n'est plus le meme : les couts d'erreur explosent, les biais deviennent critiques, la gouvernance monte. Adapte toujours la profondeur de ta demarche a l'impact. Un modele jouet sur un CSV public n'exige pas la meme revue qu'un score qui bloque un credit.

Donnees : le personnage principal

Les algorithmes changent. Les principes de donnees restent : definition claire, representativite, fraicheur, droits, documentation, absence de fuite, inspection des cas bizarres. Passe plus de temps sur les donnees que sur le shopping d'algorithmes. C'est le conseil le moins glamour et le plus rentable du livre. Un arbre simple sur des features excellentes bat une usine a gaz sur un tableau sale.

Mise en production (aperçu)

Un notebook n'est pas un produit. En production, tu dois gerer des entrees manquantes nouvelles, des categories inconnues, des delais, des journaux, des versions, des rollback, des alertes si la metrique chute. Tu n'as pas a tout construire aujourd'hui. Tu dois savoir que ca existe, pour ne pas crier victoire trop tot apres un score de validation. Prevour des le jour 1 un chemin "humain si doute".

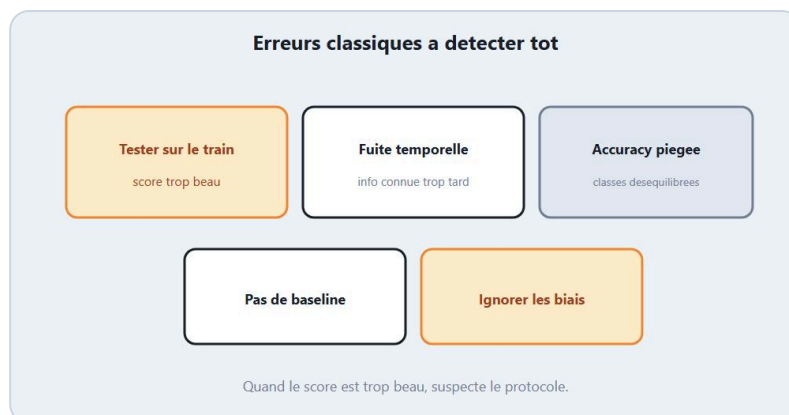
Culture et communication

Apprends a dire "non" a un modele inutile. Apprends a dire "pas encore" quand les labels manquent. Apprends a dire "voici les limites" quand tu presentes un score. Cette honnetete te rend plus credible que n'importe quel jargon. Chez DanielCraft, on forme des gens capables de tenir cette conversation avec un metier, un manager, ou un client.

Pour aller plus loin sans te perdre

Tu n'as pas a tout maitriser d'un coup. Reviens a ce chapitre quand tu butes sur un cas reel. Relis l'erreur classique. Refais le "A toi". Chez DanielCraft, on prefere trois relectures actives a une lecture passive de vingt pages. Si un paragraphe te semble encore flou, reformule-le a voix haute avec ton propre exemple. Des que ca passe a l'oral, c'est que c'est entre. Ensuite seulement, passe au chapitre suivant. Cette discipline lente cree une competence rapide sur la duree - le contraire du binge de tutos oublies le lendemain.

Chapitre 18 - Erreurs classiques (et comment les voir venir)



Erreurs classiques : train, fuite, accuracy, biais.

Voici une galerie des pieges que presque tout le monde touche une fois. L'objectif n'est pas la honte. C'est la detection precoce. Tu vas reconnaitre des symptomes : score trop beau, protocole flou, **features** interdites, metrique a cote du sujet. Ensuite tu sauras ralentir avant de deployer.

Lis cette page comme une checklist de secours. Quand un resultat te surprend (en bien ou en mal), reviens ici avant d'ajouter un algorithme. Chez DanielCraft, on dit souvent : le piege n'est pas d'etre debutant, c'est de celebrer trop tot.

★ A RETENIR

Quand le score est trop beau, suspecte d'abord le protocole (split, fuite, metrique), pas ton genie.

Les hits

Tester sur le **train**. Fuite temporelle. **Accuracy** sur classes desequilibrees. Features indisponibles en prod. IDs comme features. Optimiser jusqu'a epuiser le jeu **test**. Confondre correlation et causation. Deployer sans **baseline**. Ignorer les **biais** de sous-groupes. Notebook de recherche != script de prod. Metrique jolie, decision metier absente. Sur-complexifier jour 1.

Comment se rattraper

Revenir au contrat : question, moment de prediction, split, baseline, metrique. Reinspecter 20 erreurs. Simplifier. Mesurer par segment. Documenter les limites. Parfois le projet s'arrete : "pas assez de signal" est un resultat honnete.

♦ ASTUCE

Garde une liste de 3 questions anti-demo : split ? baseline ? sous-groupes ? Pose-les a chaque score spectaculaire (le tien inclus).

Mini histoires

Noe a cru tenir un modele de retour a 97 % d'accuracy... en predictant presque toujours "pas de retour" sur un jeu a 8 % de positifs. Lea data (version ML) a scale sur tout le CSV avant split et a fete un score irreel. Sam a utilise la note finale pour predire le risque de decrochage... note connue trop tard. Trois lecons, un meme reflexe : ralentir et verifier le protocole.

⚠ ATTENTION

97 % d'accuracy avec 8 % de positifs, en predissant presque toujours "non" : le modele a gagne le score, pas le metier.

A toi

Coche les 3 erreurs que tu es le plus susceptible de faire. Ecris un antidote en une phrase pour chacune.

Exercice miroir

Prends un article ou un post qui annonce un score spectaculaire. Liste trois questions que tu poserais (split ? baseline ? sous-groupes ?). Garde cette liste pour tes propres demos.

Developpement : penser comme un artisan des modeles

Le machine learning n'est pas un distributeur de verite. C'est un artisanat de decisions sous incertitude. Tu choisis une question, tu rassembles des exemples, tu acceptes une erreur moyenne, tu te donnes les moyens de la mesurer, tu decides si cette erreur est tolerable pour le cas d'usage. Beaucoup de frustration vient d'attendre la perfection la ou il fallait un score utile avec un humain dans la boucle.

Quand Noe predit un risque de retour, il ne remplace pas le service client. Il priorise. Quand un hopital utilise un score (hors du perimetre de ce livre introductif, et avec des cadres stricts), l'enjeu n'est plus le meme : les couts d'erreur explosent, les biais deviennent critiques, la gouvernance monte. Adapte toujours la profondeur de ta demarche a l'impact. Un modele jouet sur un CSV public n'exige pas la meme revue qu'un score qui bloque un credit.

Donnees : le personnage principal

Les algorithmes changent. Les principes de donnees restent : definition claire, representativite, fraicheur, droits, documentation, absence de fuite, inspection des cas bizarres. Passe plus de temps sur les donnees que sur le shopping d'algorithmes. C'est le conseil le moins glamour et le plus rentable du livre. Un arbre simple sur des features excellentes bat une usine a gaz sur un tableau sale.

Mise en production (aperçu)

Un notebook n'est pas un produit. En production, tu dois gerer des entrees manquantes nouvelles, des categories inconnues, des delais, des journaux, des versions, des rollback, des alertes si la metrique chute. Tu n'as pas a tout construire aujourd'hui. Tu dois savoir que ca existe, pour ne pas crier victoire trop tot apres un score de validation. Prevour des le jour 1 un chemin "humain si doute".

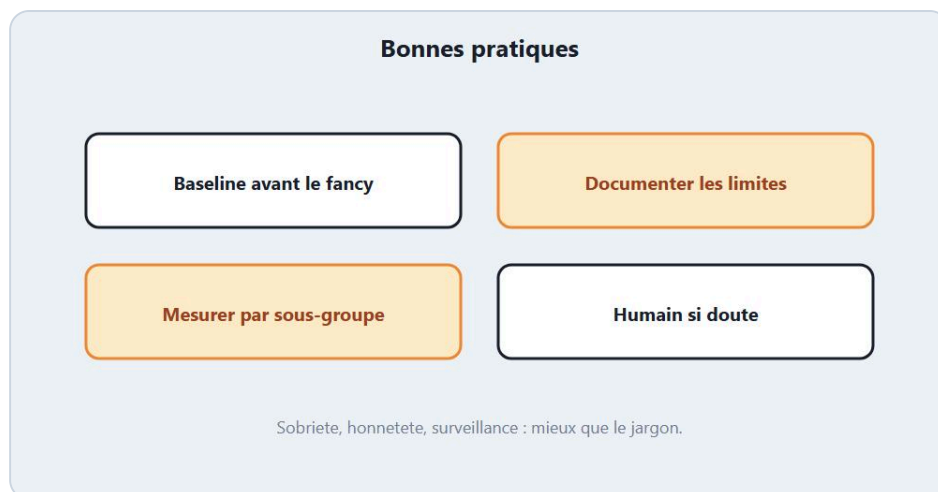
Culture et communication

Apprends a dire "non" a un modele inutile. Apprends a dire "pas encore" quand les labels manquent. Apprends a dire "voici les limites" quand tu presentes un score. Cette honnetete te rend plus credible que n'importe quel jargon. Chez DanielCraft, on forme des gens capables de tenir cette conversation avec un metier, un manager, ou un client.

Pour aller plus loin sans te perdre

Tu n'as pas a tout maitriser d'un coup. Reviens a ce chapitre quand tu butes sur un cas reel. Relis l'erreur classique. Refais le "A toi". Chez DanielCraft, on prefere trois relectures actives a une lecture passive de vingt pages. Si un paragraphe te semble encore flou, reformule-le a voix haute avec ton propre exemple. Des que ca passe a l'oral, c'est que c'est entre. Ensuite seulement, passe au chapitre suivant. Cette discipline lente cree une competence rapide sur la duree - le contraire du binge de tutos oublies le lendemain.

Chapitre 19 - Bonnes pratiques ML



Bonnes pratiques : baseline, limites, sous-groupes.

Les bonnes pratiques ne sont pas une religion. C'est une hygiène : clarifier la decision, honorer le **split**, battre une **baseline**, garder un **pipeline** propre, mesurer ce qui compte, surveiller apres coup. Si tu ne fais que ca, tu es deja devant beaucoup de demos qui enchainent les librairies.

Ce chapitre resume le style DanielCraft pour le **machine learning** du quotidien. Tu peux l'afficher, le cocher, le relire avant un go/no-go. L'idee n'est pas d'etre parfait. C'est d'etre coherent du brief jusqu'au monitoring.

★ A RETENIR

Baseline honnete + protocole clair > modele complexe mal evalue.

Avant de coder

Clarifier la decision metier. Definir y. Definir l'instant de prediction. Estimer le cout des erreurs. Verifier qu'on a (ou peut avoir) des **labels**. Preferer un projet petit qui decide vraiment a un modele "interessant" qui ne change rien.

Pendant

Split honnete. Baseline. Modele simple. Pipeline. **Metriques** alignees. Inspection des erreurs. **Features** disponibles. Mesure par sous-groupes. Journaliser les experiences (ce que tu as essaye, scores, conclusions).

♦ ASTUCE

Journalise chaque essai en trois lignes : hypothese, score, conclusion. Tu evites de retenter le meme piege la semaine suivante.

Apres

Sauvegarder artefacts + definitions. Monitoring. Plan de reentrainement. Point humain sur decisions a fort impact. Communication claire des limites aux utilisateurs du score.

Culture

Celebrer les baselines battues honnetement, pas les scores magiques. Preferer la clarte a la magie. Dire "je ne sais pas encore" quand le signal est faible.

⚠ ATTENTION

Un score sans limites documentees n'est pas pret pour un utilisateur metier. Ecris ce que le modele ne doit pas decider seul.

A toi

Transforme cette page en checklist murale de 12 cases. Coche a chaque projet.

Ritual de fin de sprint

Une heure : relire les erreurs, mettre a jour la doc features, verifier le monitoring, supprimer une experience morte, ecrire ce qu'on ne tentera plus. Les projets ML meurent souvent d'accumulation, pas de manque d'idees.

Developpement : penser comme un artisan des modeles

Le machine learning n'est pas un distributeur de verite. C'est un artisanat de decisions sous incertitude. Tu choisis une question, tu rassembles des exemples, tu acceptes une erreur moyenne, tu te donnes les moyens de la mesurer, tu decides si cette erreur est tolerable pour le cas d'usage. Beaucoup de frustration vient d'attendre la perfection la ou il fallait un score utile avec un humain dans la boucle.

Quand Noe predit un risque de retour, il ne remplace pas le service client. Il priorise. Quand un hopital utilise un score (hors du perimetre de ce livre introductif, et avec des cadres stricts), l'enjeu n'est plus le meme : les couts d'erreur explosent, les biais deviennent critiques, la gouvernance monte. Adapte toujours la profondeur de ta demarche a l'impact. Un modele jouet sur un CSV public n'exige pas la meme revue qu'un score qui bloque un credit.

Donnees : le personnage principal

Les algorithmes changent. Les principes de donnees restent : definition claire, representativite, fraicheur, droits, documentation, absence de fuite, inspection des cas bizarres. Passe plus de temps sur les donnees que sur le shopping d'algorithmes. C'est le conseil le moins glamour et le plus rentable du livre. Un arbre simple sur des features excellentes bat une usine a gaz sur un tableau sale.

Mise en production (aperçu)

Un notebook n'est pas un produit. En production, tu dois gerer des entrees manquantes nouvelles, des categories inconnues, des delais, des journaux, des versions, des rollback, des alertes si la metrique chute. Tu n'as pas a tout construire aujourd'hui. Tu dois savoir que ca existe, pour ne pas crier victoire trop tot apres un score de validation. Prevour des le jour 1 un chemin "humain si doute".

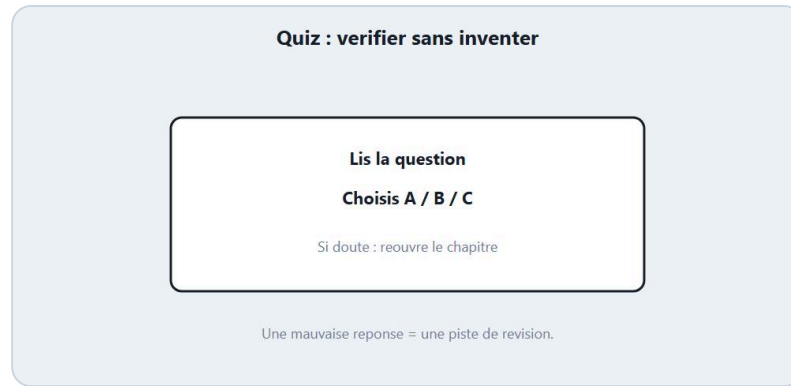
Culture et communication

Apprends a dire "non" a un modele inutile. Apprends a dire "pas encore" quand les labels manquent. Apprends a dire "voici les limites" quand tu presentes un score. Cette honnetete te rend plus credible que n'importe quel jargon. Chez DanielCraft, on forme des gens capables de tenir cette conversation avec un metier, un manager, ou un client.

Pour aller plus loin sans te perdre

Tu n'as pas a tout maitriser d'un coup. Reviens a ce chapitre quand tu butes sur un cas reel. Relis l'erreur classique. Refais le "A toi". Chez DanielCraft, on prefere trois relectures actives a une lecture passive de vingt pages. Si un paragraphe te semble encore flou, reformule-le a voix haute avec ton propre exemple. Des que ca passe a l'oral, c'est que c'est entre. Ensuite seulement, passe au chapitre suivant. Cette discipline lente cree une competence rapide sur la duree - le contraire du binge de tutos oublies le lendemain.

Quiz final



Quiz : verifier ce que tu as compris.

1. 1. L'apprentissage supervise necessite surtout :

- A) Aucun exemple
- B) Des exemples avec labels (bonnes reponses)
- C) Uniquement des images de chats

1. 2. Predire un prix, c'est typiquement :

- A) De la classification
- B) De la regression
- C) Du clustering obligatoire

1. 3. Train/test sert a :

- A) Faire joli dans un rapport
- B) Evaluer la generalisation sans se mentir
- C) Doubler la taille memoire

1. 4. L'overfitting, c'est :

- A) Coller trop au train et generaliser mal
- B) Avoir trop peu de parametres toujours
- C) Un antivirus

1. 5. Une bonne feature doit etre :

- A) N'importe quelle colonne du CSV
- B) Disponible a l'instant de prediction et definie clairement
- C) Un ID unique de preference

1. 6. Sur classes desequilibrees, l'accuracy globale :

- A) Suffit toujours
- B) Peut tromper ; regarder precision/rappel/couts
- C) Est interdite par la loi

1. 7. Un pipeline aide surtout a :

- A) Eviter fuites et incoherences de transformation
- B) Remplacer les donnees
- C) Dessiner des logos

1. 8. Les biais dans les donnees :

- A) Disparaissent magiquement dans les maths
- B) Peuvent etre herites et amplifies par le modele
- C) N'existent que dans les LLM

1. 9. Scikit-learn, en idee :

- A) Un reseau social
- B) Une bibliotheque avec une grammaire fit/transform/predict
- C) Un type de GPU

1. 10. Avant un modele complexe, DanielCraft recommande :

- A) Une baseline simple et un protocole honnete
- B) Dix librairies a la mode
- C) D'ignorer la metrique metier

Corriges

1-B, 2-B, 3-B, 4-A, 5-B, 6-B, 7-A, 8-B, 9-B, 10-A.

9/10 ou plus : solide. Sinon, relis features, train/test, metriques, biais, puis refais apres un atelier.

Bonus

Explique en cinq lignes a un ami la difference supervise / non supervise, et pourquoi l'accuracy peut mentir. Si tu bloques, relis chapitres 2, 3, 10.

Developpement : penser comme un artisan des modeles

Le machine learning n'est pas un distributeur de verite. C'est un artisanat de decisions sous incertitude. Tu choisis une question, tu rassembles des exemples, tu acceptes une erreur moyenne, tu te donnes les moyens de la mesurer, tu decides si cette erreur est tolerable pour le cas d'usage. Beaucoup de frustration vient d'attendre la perfection la ou il fallait un score utile avec un humain dans la boucle.

Quand Noe predit un risque de retour, il ne remplace pas le service client. Il priorise. Quand un hopital utilise un score (hors du perimetre de ce livre introductif, et avec des cadres stricts), l'enjeu n'est plus le meme : les couts d'erreur explosent, les biais deviennent critiques, la gouvernance monte. Adapte toujours la profondeur de ta demarche a l'impact. Un modele jouet sur un CSV public n'exige pas la meme revue qu'un score qui bloque un credit.

Donnees : le personnage principal

Les algorithmes changent. Les principes de donnees restent : definition claire, representativite, fraicheur, droits, documentation, absence de fuite, inspection des cas bizarres. Passe plus de temps sur les donnees que sur le shopping d'algorithmes. C'est le conseil le moins glamour et le plus rentable du livre. Un arbre simple sur des features excellentes bat une usine a gaz sur un tableau sale.

Mise en production (aperçu)

Un notebook n'est pas un produit. En production, tu dois gerer des entrees manquantes nouvelles, des categories inconnues, des delais, des journaux, des versions, des rollback, des alertes si la metrique chute. Tu n'as pas a tout construire aujourd'hui. Tu dois savoir que ca existe, pour ne pas crier victoire trop tot apres un score de validation. Prevour des le jour 1 un chemin "humain si doute".

Culture et communication

Apprends a dire "non" a un modele inutile. Apprends a dire "pas encore" quand les labels manquent. Apprends a dire "voici les limites" quand tu presentes un score. Cette honnetete te rend plus credible que n'importe quel jargon. Chez DanielCraft, on forme des gens capables de tenir cette conversation avec un metier, un manager, ou un client.

Pour aller plus loin sans te perdre

Tu n'as pas a tout maitriser d'un coup. Reviens a ce chapitre quand tu butes sur un cas reel. Relis l'erreur classique. Refais le "A toi". Chez DanielCraft, on prefere trois relectures actives a une lecture passive de vingt pages. Si un paragraphe te semble encore flou, reformule-le a voix haute avec ton propre exemple. Des que ca passe a l'oral, c'est que c'est entre. Ensuite seulement, passe au chapitre suivant. Cette discipline lente cree une competence rapide sur la duree - le contraire du binge de tutos oublies le lendemain.

Chapitre 21 - Bravo



Bravo. Tu as les bases du machine learning.

Tu sais maintenant ce qu'est le machine learning, comment on apprend avec ou sans labels, comment on prédit un nombre ou une classe, comment un arbre pose des questions, pourquoi le split train/test est sacré, comment l'overfitting trompe, comment choisir des features et des metriques, comment penser un pipeline, comment les biais se glissent dans les données, et quelle est l'idée de scikit-learn.

Ce n'est pas la fin de la route data. C'est une tête bien faite pour ne pas te faire avoir par un score. Chez DanielCraft, on préfère ça à la collection de buzzwords.

La suite

Pratique sur un petit CSV réel. Garde la boucle baseline -> simple -> mesurer. Si tu veux comprendre les réseaux de neurones et le lien avec les LLM, enchaîne sur le livre Deep Learning du même parcours.

Bravo. Maintenant, allez mesurer une baseline - sans tricher.

Projet 30 jours

Semaine 1 : contrat + données. Semaine 2 : baseline + split. Semaine 3 : modèle simple + inspection. Semaine 4 : pipeline + doc limites. Livre un paragraphe "ce que le modèle ne doit pas décider seul".

Developpement : penser comme un artisan des modeles

Le machine learning n'est pas un distributeur de vérité. C'est un artisanat de décisions sous incertitude. Tu choisis une question, tu rassembles des exemples, tu acceptes une erreur moyenne, tu te donnes les moyens de la mesurer, tu décides si cette erreur est tolérable pour le cas d'usage. Beaucoup de frustration vient d'attendre la perfection là où il fallait un score utile avec un humain dans la boucle.

Quand Noe prédit un risque de retour, il ne remplace pas le service client. Il priorise. Quand un hôpital utilise un score (hors du périmètre de ce livre introductif, et avec des cadres stricts), l'enjeu n'est plus le même : les coûts d'erreur explosent, les biais deviennent critiques, la gouvernance monte. Adapte toujours la profondeur de ta démarche à l'impact. Un modèle jouet sur un CSV public n'exige pas la même revue qu'un score qui bloque un crédit.

Donnees : le personnage principal

Les algorithmes changent. Les principes de donnees restent : definition claire, representativite, fraicheur, droits, documentation, absence de fuite, inspection des cas bizarres. Passe plus de temps sur les donnees que sur le shopping d'algorithmes. C'est le conseil le moins glamour et le plus rentable du livre. Un arbre simple sur des features excellentes bat une usine a gaz sur un tableau sale.

Mise en production (aperçu)

Un notebook n'est pas un produit. En production, tu dois gerer des entrees manquantes nouvelles, des categories inconnues, des delais, des journaux, des versions, des rollback, des alertes si la metrique chute. Tu n'as pas a tout construire aujourd'hui. Tu dois savoir que ca existe, pour ne pas crier victoire trop tot apres un score de validation. Prevour des le jour 1 un chemin "humain si doute".

Culture et communication

Apprends a dire "non" a un modele inutile. Apprends a dire "pas encore" quand les labels manquent. Apprends a dire "voici les limites" quand tu presentes un score. Cette honnetete te rend plus credible que n'importe quel jargon. Chez DanielCraft, on forme des gens capables de tenir cette conversation avec un metier, un manager, ou un client.

Pour aller plus loin sans te perdre

Tu n'as pas a tout maitriser d'un coup. Reviens a ce chapitre quand tu butes sur un cas reel. Relis l'erreur classique. Refais le "A toi". Chez DanielCraft, on prefere trois relectures actives a une lecture passive de vingt pages. Si un paragraphe te semble encore flou, reformule-le a voix haute avec ton propre exemple. Des que ca passe a l'oral, c'est que c'est entre. Ensuite seulement, passe au chapitre suivant. Cette discipline lente cree une competence rapide sur la duree - le contraire du binge de tutos oublies le lendemain.

Tu as les briques. Maintenant, construis.